西南交通大学
Southwest Jiaotong University

《算法分析与设计B》

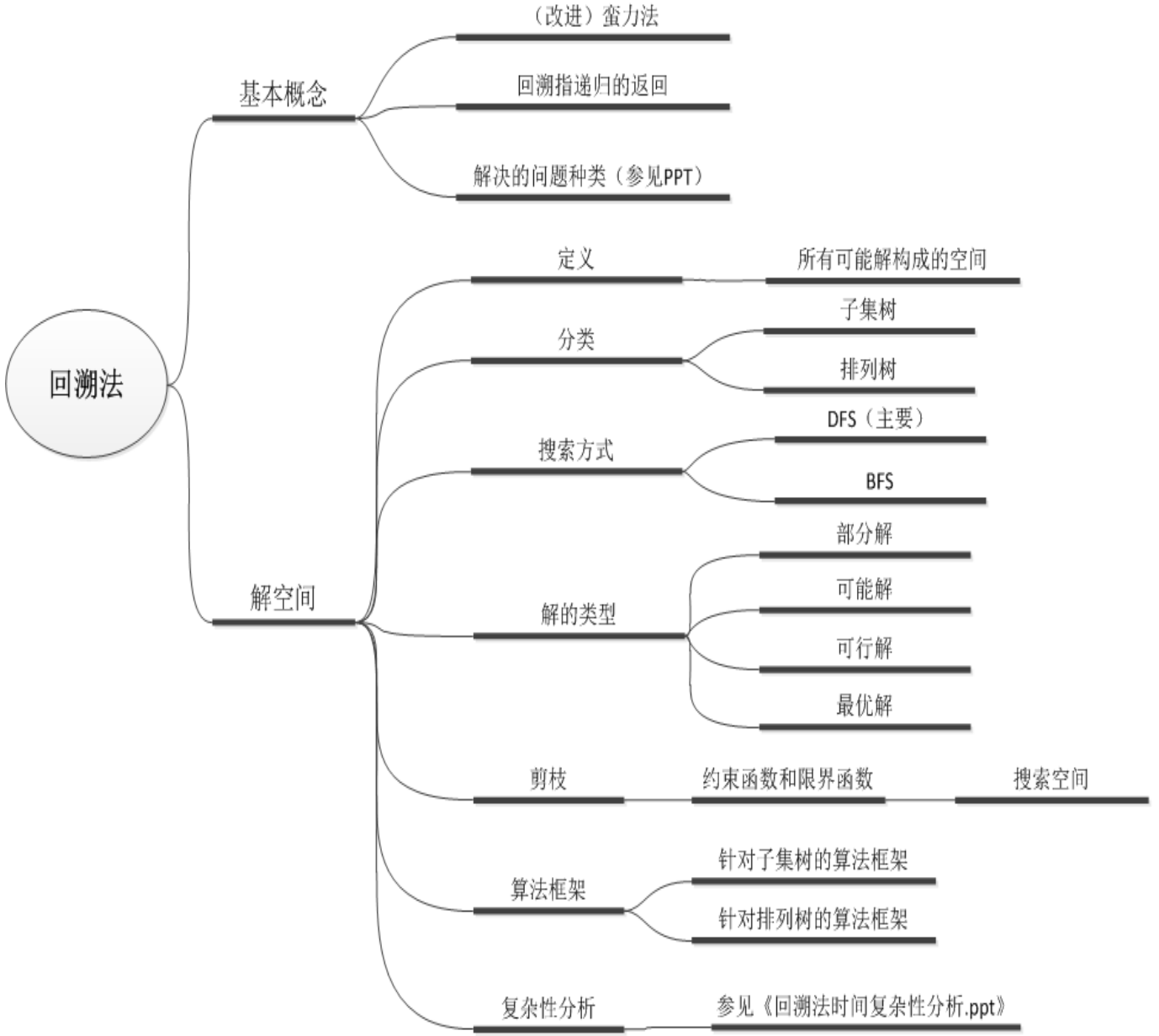
回溯法和分支限界法

计算机与人工智能学院
李晓航
xhli_scce@swjtu.edu.cn

Last: 2026-03

一、回溯法概述

- 1.1 回溯法基本思路
- 1.2 回溯法相关概念
- 1.3 问题的解空间
- 1.4 回溯法基本步骤
- 1.5 回溯法算法框架
- 1.6 回溯法小结



1.1 回溯法基本思路

- 回溯法（BackTracking）和分支限界法是在蛮力法的基础上的一个改进。
- 蛮力法是先搜索所有的可能解，然后再评估这些可能解是否满足约束条件
  - 理论上，当可能解数量有限并且通过检查所有或部分可能解能够得到所需解时，蛮力法是可行的。
  - 不过在实际应用中，很少使用蛮力法，因为可能解的数量通常都非常大（如指数级或阶乘级）。
- 而回溯法是避免了不必要搜索的穷举式搜索法，该方法适用于解一些组合数相当大的问题。

回溯法基本思路（续）

- 为了实现回溯，首先需要为问题定义一个解空间（通常组织成树形结构，即解空间树），这个空间必须至少包含问题的一个解。
- 回溯法在包含问题所有解的解空间树中，按深度优先策略，从根结点出发搜索解空间树
  - 算法搜索至解空间树的任意一结点时，先判断该结点是否包含问题的解。
  - 如果肯定不包含，则跳过对该结点为根的子树的搜索，逐层向其祖先结点回溯；否则，进入该子树，继续按深度优先策略搜索。
- 从构造问题解的角度看
  - 回溯法（和分支限界法）每次只构造可能解的一部分（部分解），然后评估这个部分解。
  - 如果这个部分解有可能导致一个可行解，则对其进一步构造，否则就不必继续构造这个部分解，从而避免搜索所有的可能解。

回顾：深度优先（DFS）和广度优先（BFS）



深度优先：先访问A节点，再访问B节点，然后访问E节点，E节点没有子节点，回溯到B节点，然后访问F节点，F节点没有子节点，回溯到B节点，B节点没有其他未访问的子节点，回溯到A节点，再访问C节点。D、G、H节点同理，最后访问完整棵树。

最终结果：A->B->E->F->C->D->G->H

广度优先：先访问A节点，然后访问A节点的所有子节点，即B、C、D节点，再访问B节点的所有子节点E、F节点，D节点的所有子节点G、H节点。

最终结果：A->B->C->D->E->F->G->H

1.2 回溯法相关概念

- 回溯法是一种通过搜索解空间树（状态空间树）来求问题的可行解或最优解的方法
  - 可能解：解空间树中的所有解（不一定满足问题的约束条件）。
  - 可行解：满足约束条件的解。
  - 最优解：使最优化问题的目标函数取得极值（最大值或最小值）的解。
- 目标函数和最优化问题
  - 目标函数：满足约束条件的可行解可能不只一个，为了衡量这些可行解的优劣，事先给出一定的标准，这些标准通常以函数的形式给出，这些函数称为目标函数（也叫优化函数）。
  - 最优化问题：使目标函数取得极值（极大或极小）的可行解称为最优解，这类问题就称为最优化问题。

回溯法解决的问题类型

- 组合问题：
  - 如N个数里面按一定规则找出k个数的集合
- 子集问题：
  - 如一个N个数的集合里有多少符合条件的子集
- 排列问题：
  - 如N个数按一定规则全排列，有几种排列方式
- 棋盘问题：
  - 如N皇后，解数独，走迷宫等等
- 切割问题：
  - 如一个字符串按一定规则有几种切割方式

这类问题都可以采用**分步求解**策略，且每一步的决策都可以在一个有限集合内选择

回溯法解决的问题类型（续）

问题类型	分步选择过程
排列组合	每次从剩余元素中选一个
子集问题	每个元素选或不选
棋盘/迷宫	每个位置决定下一步方向
路径/连通性	每次扩展一个节点
数独、N皇后	逐行（或逐格）放置，每步选一个合法值

1.3 问题的解空间和解空间树

- 复杂问题常常有很多的可能解，这些可能解构成了问题的解空间
  - 解空间也就是进行穷举的搜索空间，所以，解空间中应该包括所有的可能解。
  - 为了用回溯法求解一个规模为 n 的问题，通常将其可能解表示为满足某个约束条件的等长向量$X=(x_1,x_2,...x_n)$的形式，其中分量 x_i 的取值范围是某个有限集合 $S_i=\{a_{i1},a_{i2},...a_{ir}\}$（即 x_i 可能的选择范围）。所有可能的解向量构成了问题的解空间。
  - 确定正确的解空间很重要，如果没有确定正确的解空间就开始搜索，可能会增加很多重复解，或者根本就搜索不到正确的解。

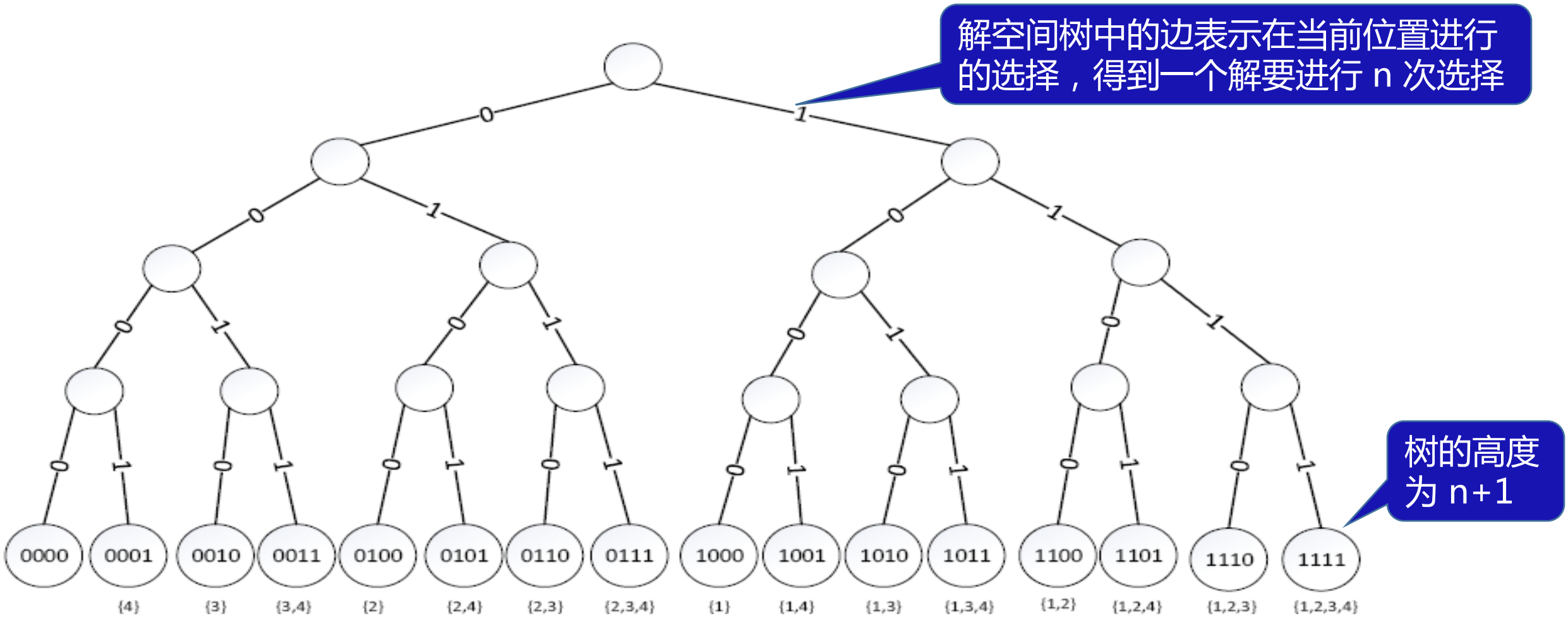
问题的解空间和解空间树

- 问题的解空间一般用解空间树（Solution Space Trees,也称状态空间树）的方式组织
  - 用树的方式组织解空间便于用回溯法对其进行搜索。
  - 树的根结点位于第 1 层，表示搜索的初始状态，第 2 层的结点表示对解向量的第一个分量做出选择后到达的状态。
  - 第 1 层到第 2 层的边上标出对第一个分量选择的结果，依此类推。从根结点到叶子结点的路径就构成了解空间的一个可能解。

两种解空间树

- 在用回溯法求解问题时，常常遇到两种典型的解空间树。
- 子集树（Subset Trees）：
  - 当所给问题是从 n 个元素的集合中找出满足某种性质的子集时，相应的解空间树称为子集树。
  - 在子集树中，$|S_1|=|S_2|=\ldots=|S_n|=m$，即每个结点有相同数目的子树，共有 m^n 个叶子节点（如果 m=2，则有 2^n 个叶子结点）。
  - 子集树的节点总数为：$(m^{n+1}-1)/(m-1)$（当m=2时为：$2^{n+1}-1$）。
- 排列树（Permutation Trees）：
  - 当所给问题是确定 n 个元素满足某种性质的排列时，相应的解空间树称为排列树。
  - 在排列树中，通常情况下，$|S_1|=n$，$|S_2|=n-1$，…，$|S_n|=1$，所以，排列树中共有(n-1)!个叶子结点。
  - 排列树的节点总数为：$(1+n!/(n-1)!+ n!/(n-2)! + n!/1!+n!/0!) < 4*n!$。

m=2，n=4的子集树（二叉树）

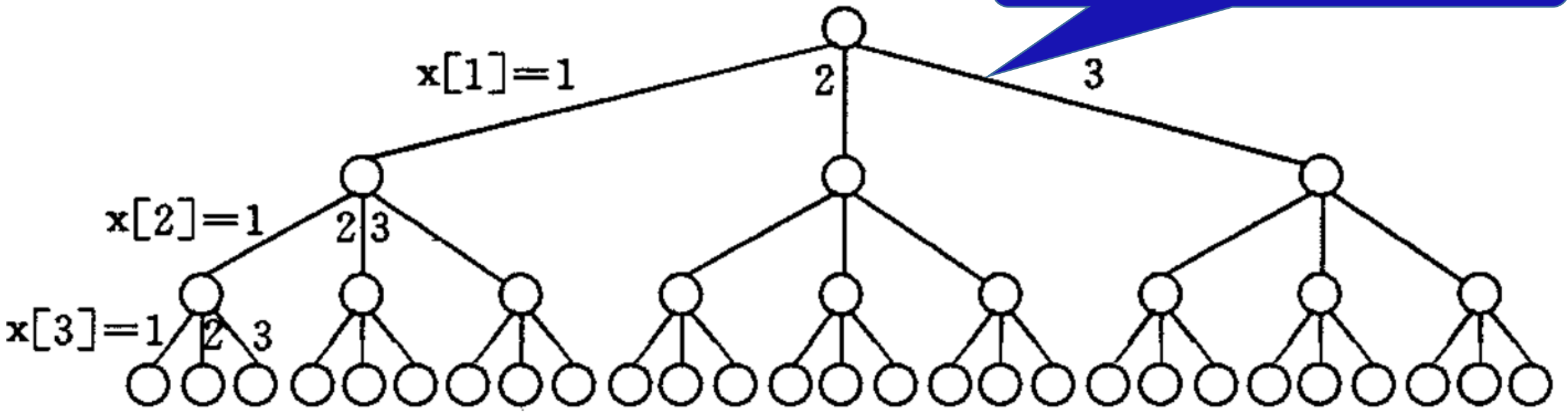


子集树中，每个结点有相同数目的子树。

该子集树的节点总数为 $2^0+2^1+\ldots+2^n = 2^{n+1}-1$（节点总数可用于计算时间复杂性）

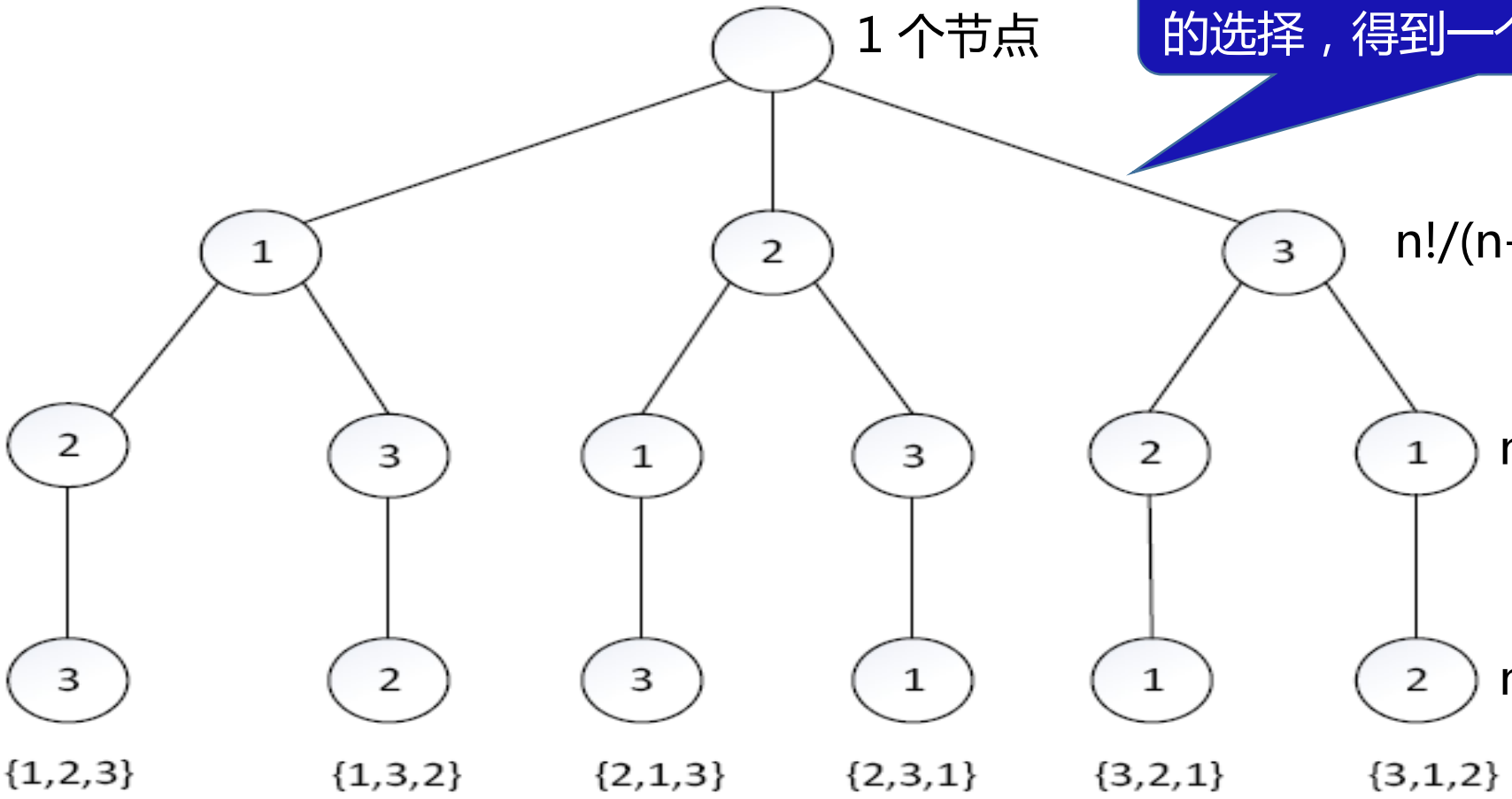
m>2（m=3，n=3）的子集树（m叉树）

解空间树中的边表示在当前位置进行的选择，得到一个解要进行 n 次选择



该子集树的节点总数为 $3^0+3^1+\ldots+3^n = (3^{n+1}-1)/(3-1)$

n=3 的全排列树



解空间树中的边表示在当前位置进行的选择，得到一个解要进行 n 次选择

n!/(n-1)!=3!/2! =3 个节点

n!/(n-2)! = 3!/1! =6个节点

n!=3! =6 个节点

排列树中，每层结点的子树数目从 n 开始依次减一。

排列树的节点总数为 **(1+n!/(n-1)!+ n!/(n-2)! + n!/1!+n!/0!) < 4*n!**

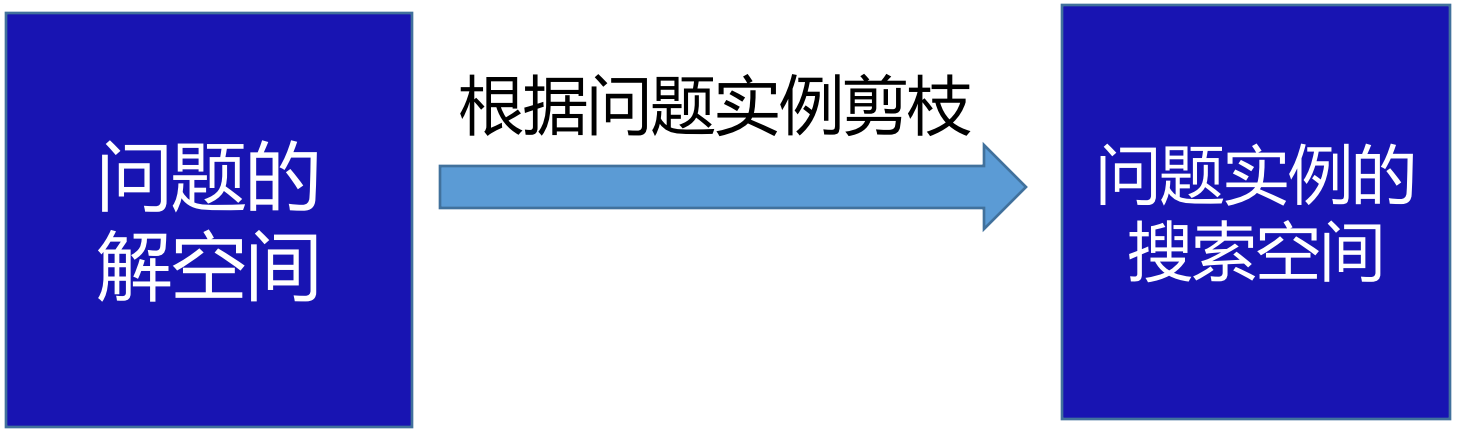
（节点数可用于计算时间复杂性）。

1.4 回溯法基本步骤

- 回溯法的基本步骤
  - 1、针对所给问题，定义问题的解空间；
  - 2、确定易于搜索的解空间结构（通常是 n 维向量）；
  - 3、以深度优先方式搜索解空间，并在搜索过程中用剪枝函数避免无效搜索。
- 剪枝函数
  - 包括约束函数（constraint）和限界函数（bound）。
- 经过剪枝后的解空间为实际的搜索空间。

解空间和搜索空间

- 解空间
  - 包含问题的所有可能解，与具体输入无关。
- 搜索空间
  - 经过剪枝后的实际搜索空间，与问题的具体实例有关。



约束函数和限界函数

- 约束函数（constraint）
  - 约束函数根据问题的定义实现，用于在扩展结点处剪去不满足约束条件的子树。
- 限界函数（bound）
  - 限界函数仅用于“最优化”问题（求最大值或最小值），通过限界函数可以剪去得不到最优解的子树。
  - 限界函数的值 = 目标函数在当前节点已经获得的值+从该节点往下能获得的理论上的最优值（一般利用贪心法计算）。
  - 最大值问题的限界函数计算理论的上界；最小值问题的限界函数计算理论的下界。

1.5 回溯法算法框架（递归算法）

```
void backtrack (int t)   { //t：递归深度
      if (t>n) output(x);   // 到达叶子结点，将结果输出
      else
          for (int i=f(n,t);  i<=g(n,t);  i++)  { // 遍历结点t的所有子结点
              x[t]=h(i);  //记录部分解
              if ( constraint(t) && bound(t) )   {
                   //处理本层状态
                  backtrack(t+1);  //递归处理下一层
                   //撤销处理结果
              }
      }
}
```

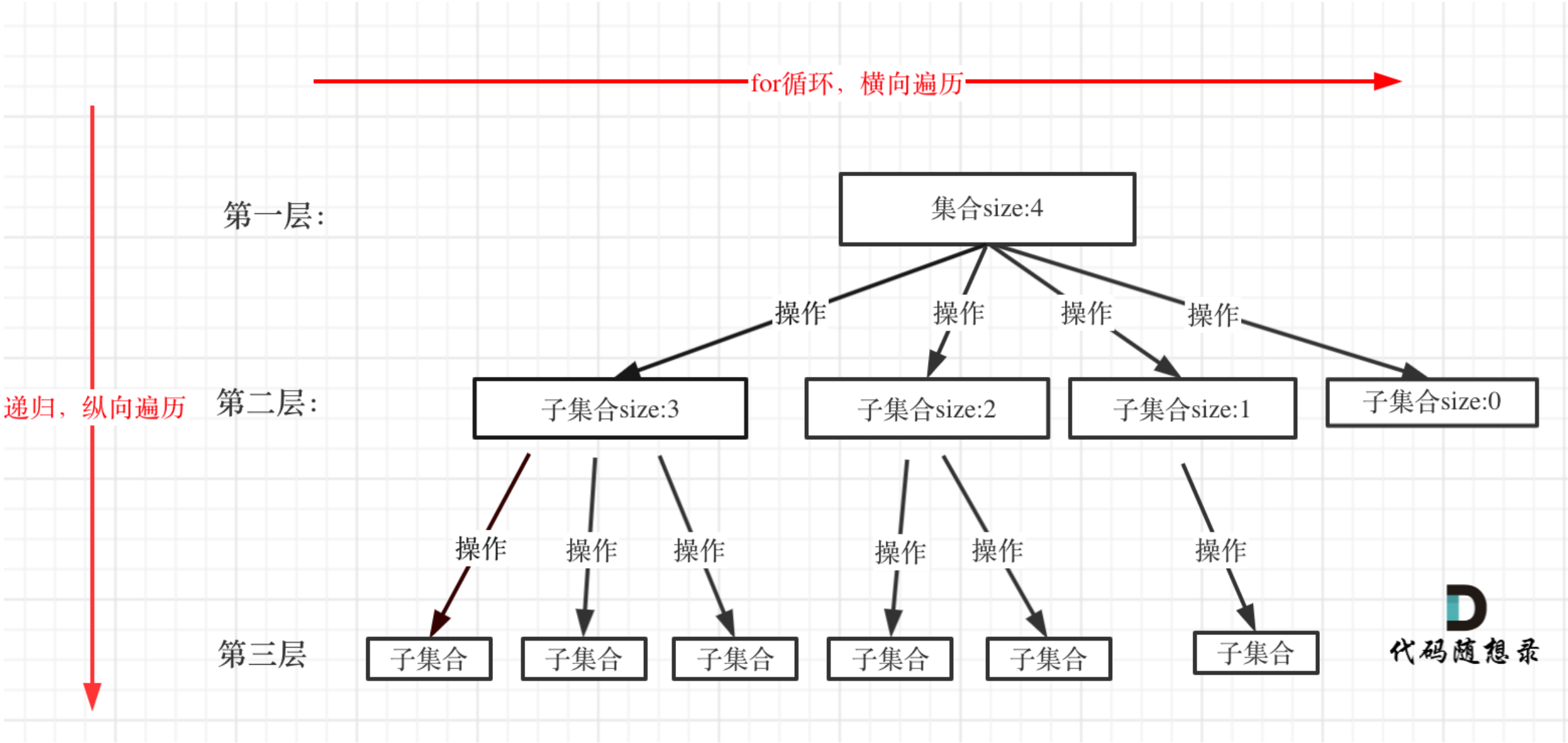
- t：递归深度，即当前扩展结点在解空间树中的深度。
- n：用来控制递归深度，即解空间树的高度。
- f(n,t)和g(n,t)：表示在当前扩展结点处未搜索过的子树的起始编号和终止编号。
- h(i)：表示在当前扩展结点处 x[t] 的第 i 个可选值，x[]表示当前部分解。
- constraint(t)和bound(t)：表示在当前扩展结点处的约束函数和限界函数。

回溯法算法框架（续）

《代码随想录》中的回溯算法框架：

```
void backtracking(参数) {   //常见的参数是递归深度
    if (终止条件) {
        存放结果;
        return;
    }
    for (选择：本层集合中元素（树中节点孩子的数量就是集合的大小））{
        处理节点（如记录部分解）;
        backtracking(参数);     // 递归（深度+1）
        回溯（撤销处理结果）；
    }
}
```

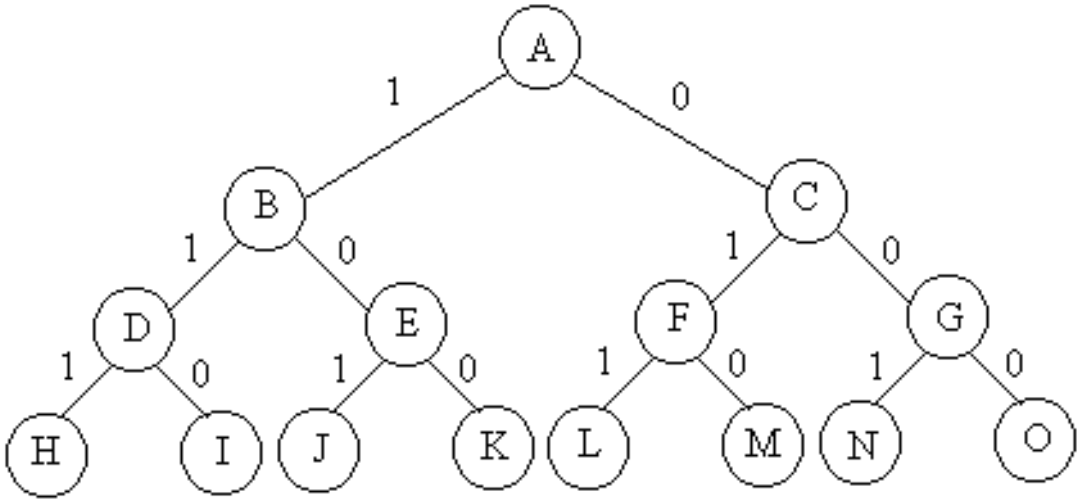
回溯算法的遍历过程


for循环，横向遍历
递归，纵向遍历
第一层：
第二层：
第三层
集合size:4
操作
操作
操作
操作
子集合size:3
子集合size:2
子集合size:1
子集合size:0
操作
操作
操作
操作
操作
操作
子集合
子集合
子集合
子集合
子集合
子集合
代码随想录

“子集树”的递归回溯

- 从n个元素的集合中找出满足某种性质的子集，相应的解空间树称为子集树。
- 子集树通常有2^n个叶结点。遍历子集树需$O(2^n)$计算时间。

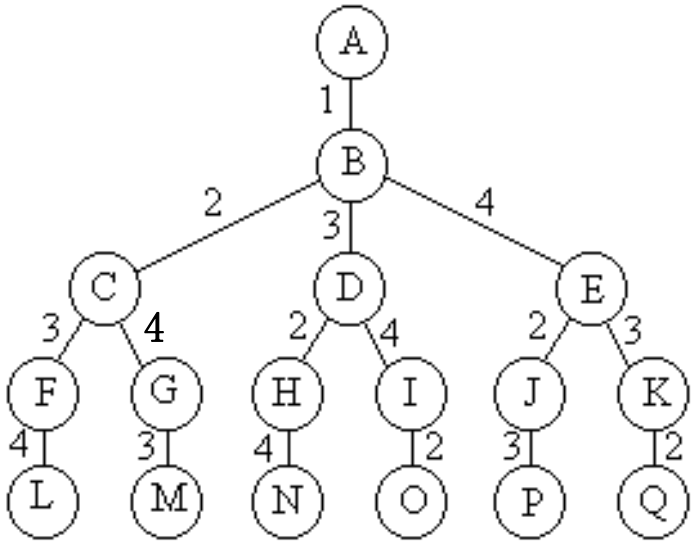
```
void backtrack (int t) //t：递归深度
{
  if (t>n) output(x);  // 到达叶子结点
  else
      for (int i=0; i<=1;i++) {   //0、1两种选择
          x[t]=i;          //记录部分解
          if (constraint(t)&&bound(t))
              backtrack(t+1);
      }
}
```



“排列树”的递归回溯

- 要确定n个元素的满足某种性质的排列时，相应的解空间树称为排列树。
- 排列数通常有(n-1)!个叶结点。因此遍历排列树需要O(n!)计算时间。

```
void backtrack (int t) //t：递归深度
{
  if (t>n) output(x);
  else
      for (int i=t; i<=n; i++) {
          swap(x[t], x[i]); // 更换排列顺序（第一个元素与后面元素交换）
          if (constraint(t)&&bound(t))
              backtrack(t+1);
          swap(x[t], x[i]); //搜索完毕后恢复
      }
}
```



“排列树”递归回溯的另一种代码框架（参见《代码随想录》），可读性更好

```
class Solution {
public:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking (vector<int>& nums, vector<bool>& used)  {
        if (path.size() == nums.size()) {     // 找到了一组排列（path.size表示递归深度）
            result.push_back(path);
            return;          }
        for (int i = 0; i < nums.size(); i++)  {
            if (used[i] == true) continue;    // 第 i 个数字已经被占用，直接跳过
            used[i] = true;
            path.push_back(nums[i]);
            backtracking(nums, used);
            path.pop_back();
            used[i] = false;         }
    }
    vector<vector<int>> permute(vector<int>& nums) {
        vector<bool> used(nums.size(), false); //used数组表示每个数字是否被占用
        backtracking(nums, used);
        return result;
    }
};
```

1.6 回溯法小结

[解空间树构造]

1.子集树: 当解向量为不定长 n 元组时 ，树中从根至每一点的路径集合构成解空间。树的每个结点称为一个解状态，有儿子的结点称为可扩展结点，叶结点称为终止结点， 若结点 v 对应解状态(x_1, x_2,... x_i)，则其儿子对应扩展的解状态 (x_1, x_2,... x_i, x_{i+1})。

2.排序树: 当解向量为定长 n 元组时, 树中从根至叶结点的路径的集合构成解空间。树的每个叶结点称为一个解状态。

[搜索过程]

搜索按深度优先策略从根开始， 当搜索到任一结点时,判断该点是否满足约束条件D(剪枝函数)， 满足则继续向下深度优先搜索，否则跳过该结点以下的子树(剪枝)， 向上逐级回溯。

[求解步骤]

1) 针对所给问题,定义问题的解空间
2) 确定约束条件.
3) 以深度优先方式搜索解空间.

二、常见的回溯法算法

- 1、0/1 背包问题
- 2、旅行商问题
- 3、n 皇后问题
- 4、装载问题
- 5、图的着色问题
- 6、子集和问题
- 7、汉密尔顿回路问题（参见教材）

2.1 回溯法解0/1背包问题

- 0/1背包问题描述
  - 给定 n 个重量为 $\{w_1, w_2, \ldots, w_n\}$、价值为 $\{v_1, v_2, \ldots, v_n\}$ 的物品和一个容量为 C 的背包，求这些物品的一个最有价值的子集，并且该子集要能够装到背包中。
- 0/1背包问题的解向量
  - 对于有 n 个物品的 0/1 背包问题，其可能解由一个等长向量 $\{x_1, x_2, \ldots, x_n\}$ 组成，x_i 为 0 或 1。其中 $x_i=1$ 表示物品 i 装入背包，$x_i=0$ 表示物品 i 不装入背包。
  - 这样可以把0/1背包问题的约束条件和目标函数表示为：

约束条件：

$$\begin{cases} \sum_{i=1}^{n} w_i x_i \le C \\ x_i \in \{0,1\} \quad (1 \le i \le n) \end{cases}$$

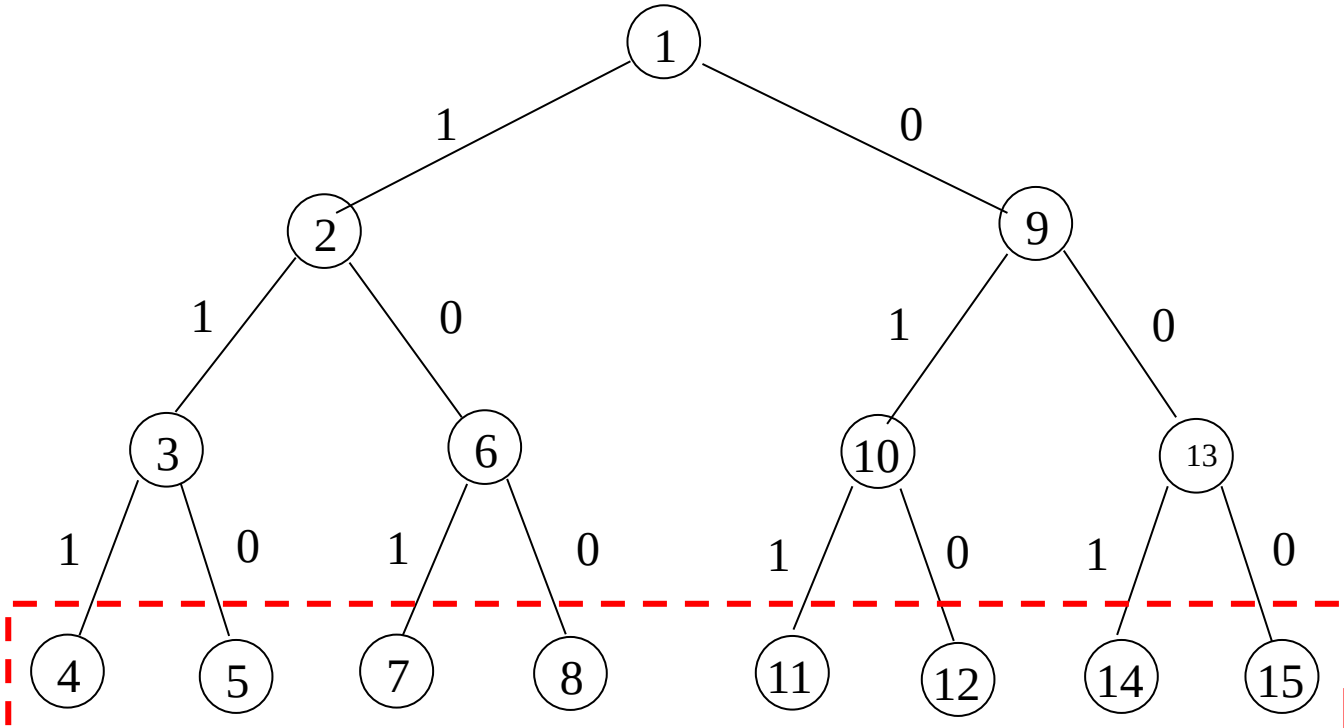
目标函数：

$$\max \sum_{i=1}^{n} v_i x_i$$

最后需要求出满足约束条件，并使目标函数成立的解向量

0/1背包问题的解空间树

- 对于n=3的 0/1 背包问题，其解空间树如下图所示（树中的 8 个叶子结点分别代表该问题的 8 个可能解向量）：



对物品1的选择

对物品2的选择

对物品3的选择

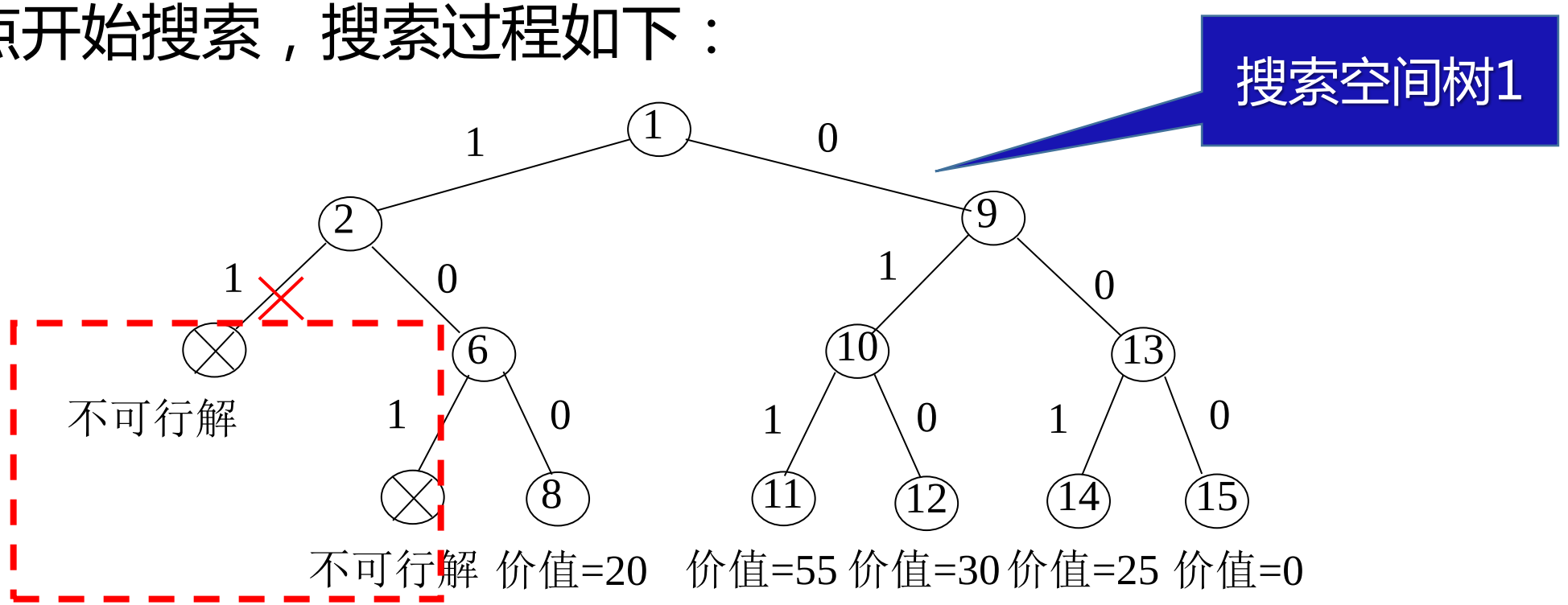
解空间树

可能解（候选解）

可能解的解向量为 (0, 0, 0), (0, 0, 1), (0, 1, 0), (1, 0, 0),
(0, 1, 1), (1, 0, 1), (1, 1, 0), (1, 1, 1)

0/1背包问题利用约束函数剪枝

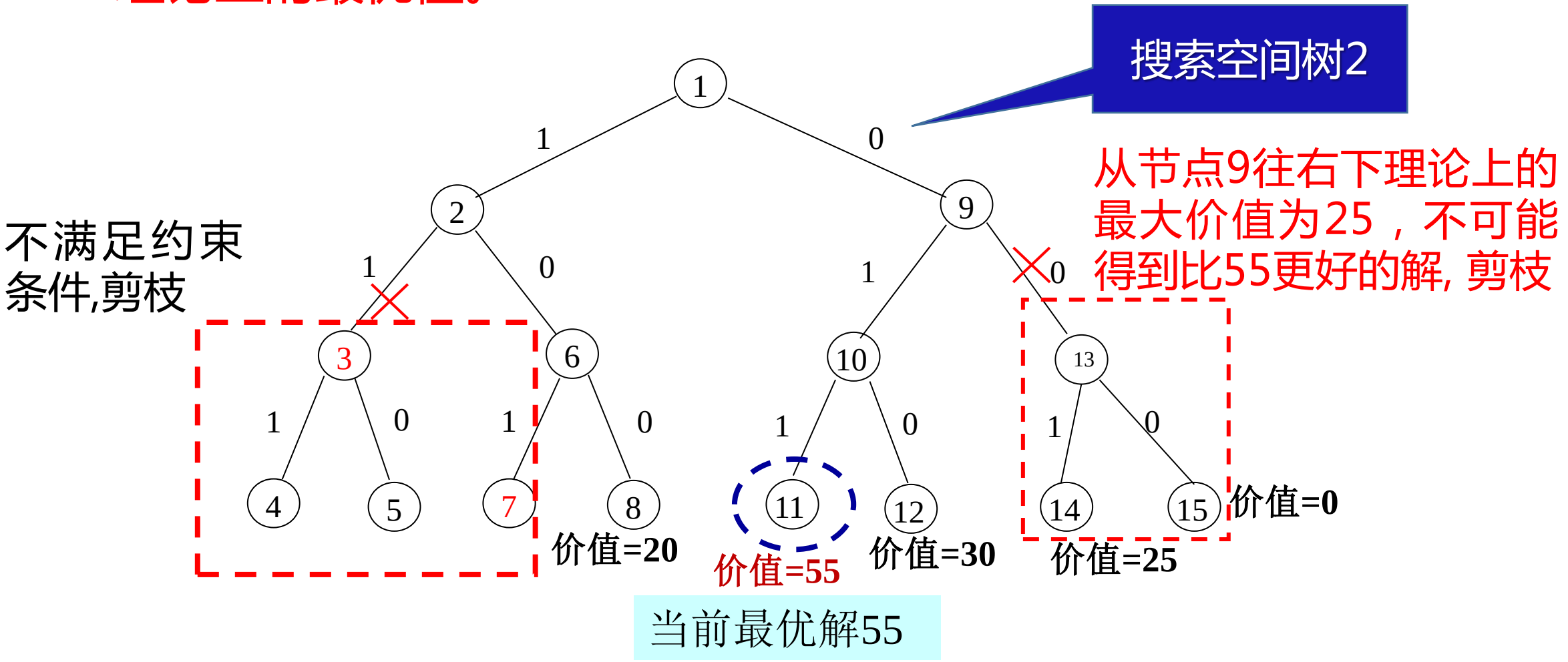
- 假设对于n=3的0/1背包问题，三个物品的重量为{20, 15, 10}，价值为{20, 30, 25}，背包容量为25，从下图所示的解空间树的根结点开始搜索，搜索过程如下：



- 为了避免生成那些不可能产生可行解的问题状态，不断利用约束函数剪枝，让那些实际上不可能产生可行解的活结点变成死结点，以减少计算量

0/1背包问题利用限界函数剪枝

- 界： 目标函数在当前节点已经获得的值+从该节点往下能获得的理论上的最优值。



回溯法解0/1背包问题代码

算法描述

解空间: 子集树

约束函数: $\sum_{i=1}^{n} w_i x_i \le c_1$

上界函数:

当前价值cv + 剩余容量可容纳的最大价值 > 当前最优价值bestv。

Double c;//背包容量
Int n;//物品数
Double [] w;//物品重量数组
Double [] v;//物品价值数组
Double cw;//当前重量
Double cv;//当前价值
Double bestv;//当前最优价值

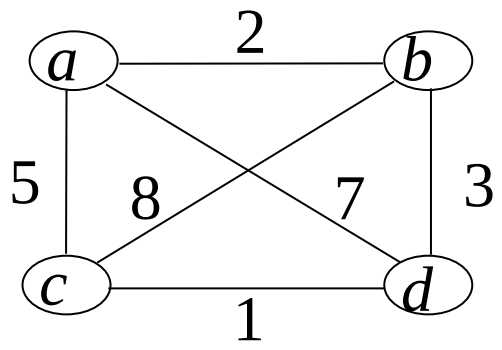
w 和 v 中的物品要按照“单位重量价值(v_i/w_i)”递减排序

```
double Bound(int i)
{// 计算上界（从 i 往下理论上能达到的最大值）
  double cleft = c - cw; // 剩余容量
  double bestv = cv;      //当前价值
  // 以“物品单位重量价值”递减序装入物品
  while (i <= n && w[i] <= cleft) {
    cleft -= w[i];
    bestv += v[i];
    i++;
  }
  // 背包仍有空隙时，装满背包（贪心法思想）
  if (i <= n) bestv += v[i]/w[i] * cleft;
  return bestv;
}
void BackTrack(int i)  //i: 递归深度
{
   if (i>n) {  if (cv>bestv) bestv=cv; return; }   //到达叶结点
   if (cw+w[i]<=c)   //满足约束函数，进入左子树
   {  cw+=w[i];    cv+=v[i];
      BackTrack(i+1);
      cw-=w[i];      cv-=v[i];
   }
   if (bound(i+1)>bestv)  //限界函数大于当前最优值
      BackTrack(i+1);      //进入右子树
}
```

2.2 旅行商问题

- 旅行商问题（TSP—Travelling Salesman Problem）问题又称为货郎问题、邮递员问题，是图问题中最广为人知的问题。
- TSP问题的描述
  - 求旅行者由起点城市出发，通过所有给定的城市之后，最后再回到起点城市的最小路径成本。
- TSP问题的解向量
  - 对于有 n 个顶点的图的TSP问题，其可能解由一个等长向量 $\{x_1, x_2, \ldots, x_n\}$ 组成，x_i 为 1 到 n（顶点编号）。解向量表示一条回路。
- TSP问题的约束条件和目标函数
  - 约束条件：“顶点之间有路径”并且“已经走过的顶点不能再走”
  - 目标函数：min(回路长度)
  - 限界函数：暂时不考虑

TSP问题的例子



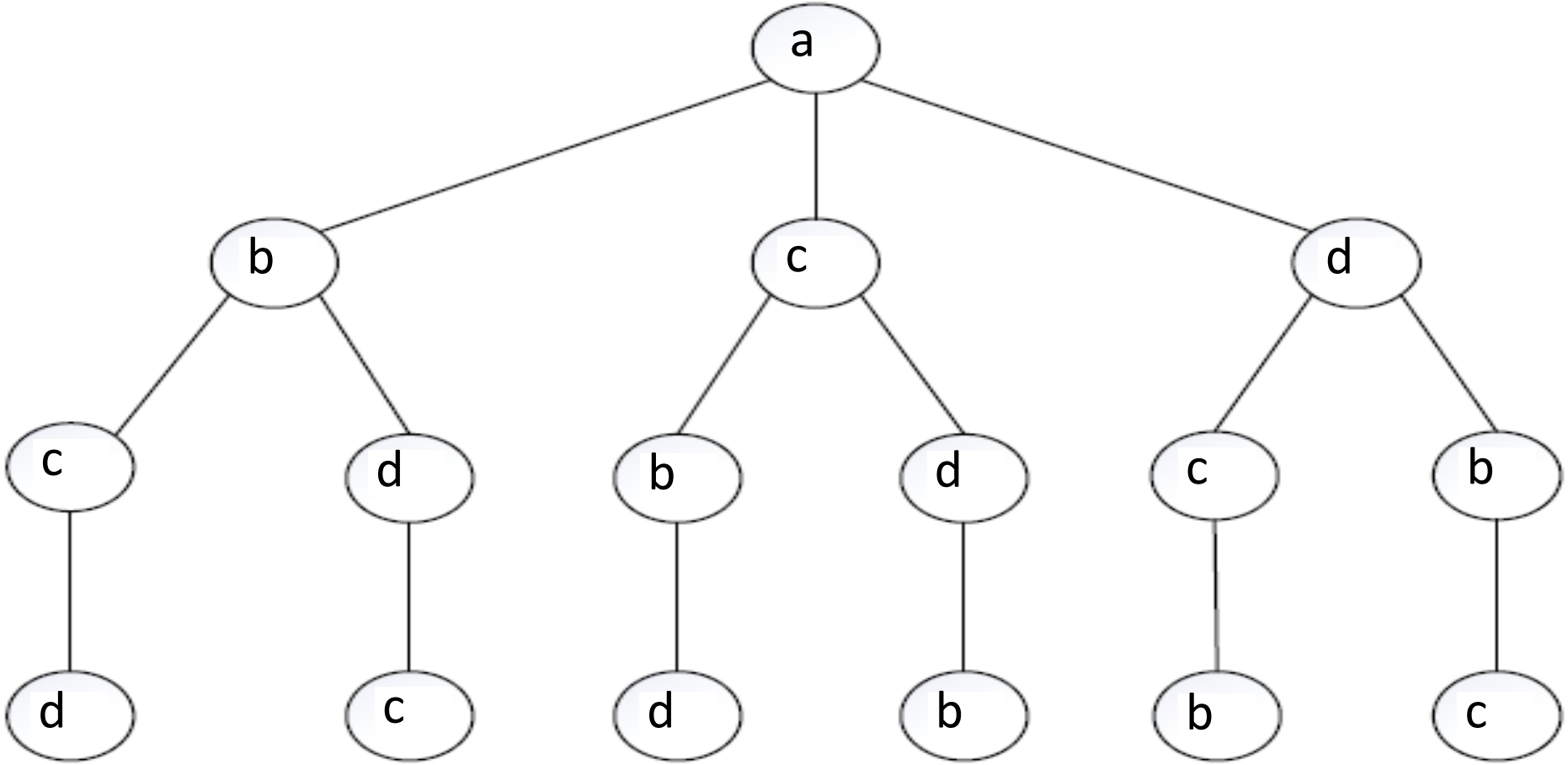
序号	路径	路径长度	是否最短
1	$a\rightarrow b\rightarrow c\rightarrow d\rightarrow a$	18	否
2	$a\rightarrow b\rightarrow d\rightarrow c\rightarrow a$	11	是
3	$a\rightarrow c\rightarrow b\rightarrow d\rightarrow a$	23	否
4	$a\rightarrow c\rightarrow d\rightarrow b\rightarrow a$	11	是
5	$a\rightarrow d\rightarrow b\rightarrow c\rightarrow a$	23	否
6	$a\rightarrow d\rightarrow c\rightarrow b\rightarrow a$	18	否
…	…	…	…

TSP问题的搜索空间：

从起点出发，遍历其余(n-1)个顶点的可能路径有 (n-1)! 条。考虑到无向图中有一对路径只是方向不同，长度相同，因此可以将搜索空间减半，即可能的解有(n-1)!/2 个。

用蛮力法求解TSP问题，其时间复杂性为O(n!)，只能解决问题规模很小的实例。

四顶点图的TSP问题的解空间树



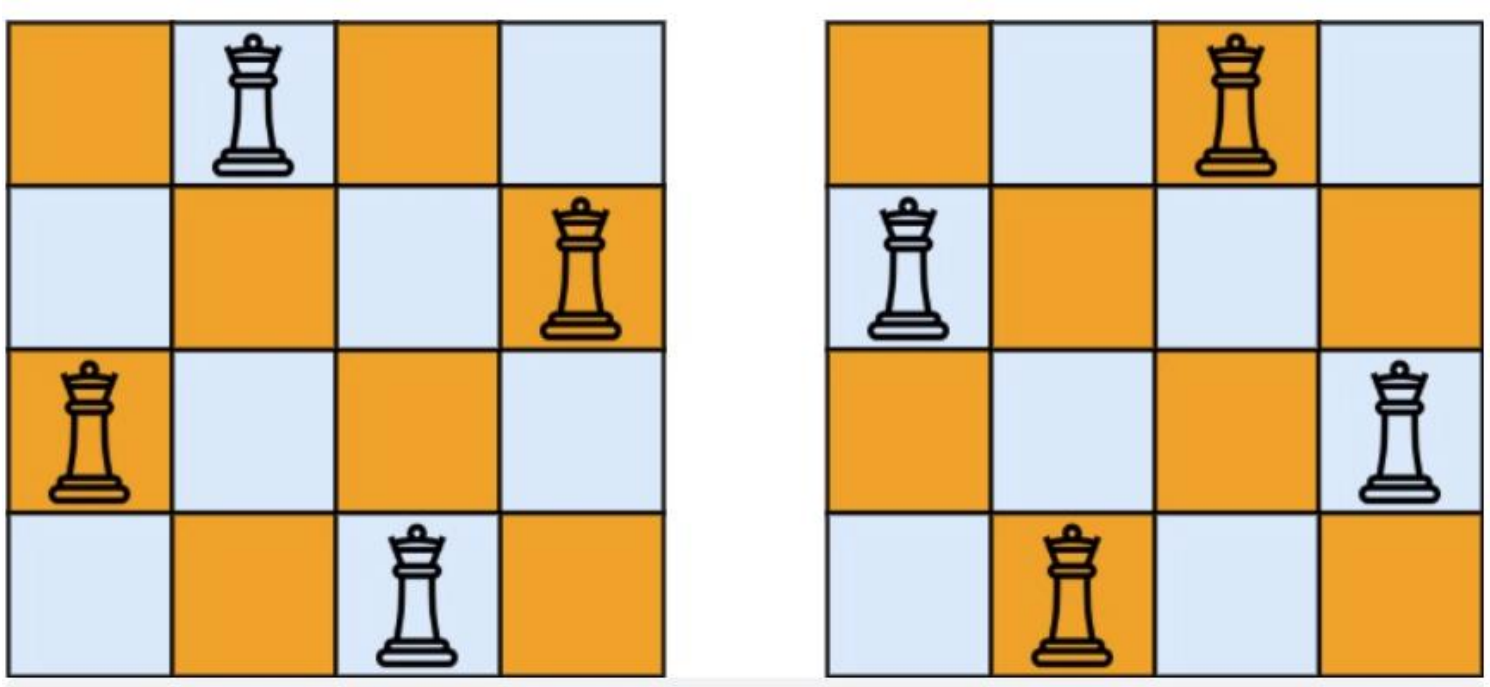
排列树

TSP问题可以通过修改排列树代码来实现

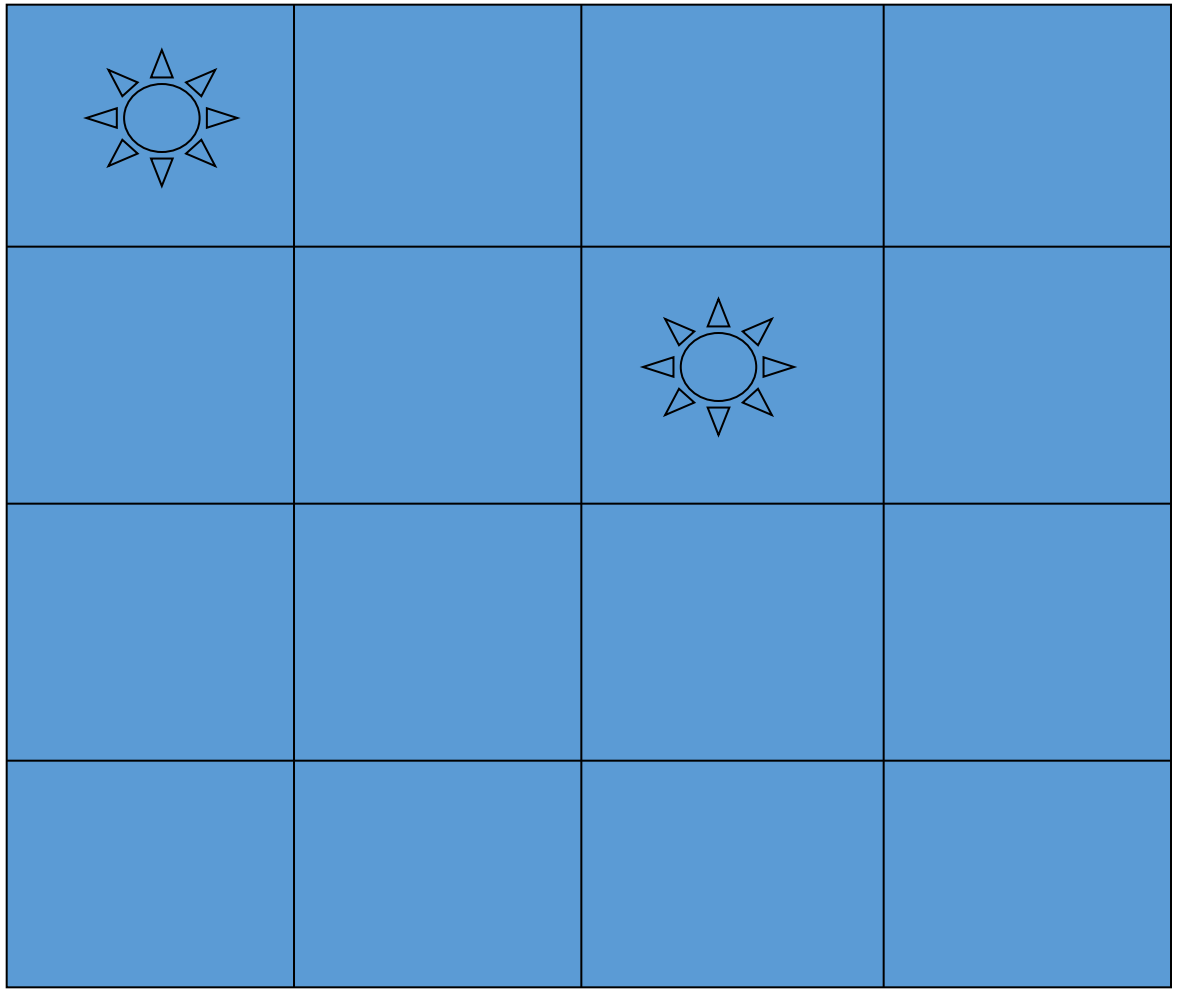
```
class Solution {
public:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking (vector<int>& nums, vector<bool>& used)  {
        if (path.size() == nums.size()) {     // 找到了一组排列（path.size表示递归深度）
            result.push_back(path);
            return;          }
        for (int i = 0; i < nums.size(); i++)  {
            if (used[i] == true) continue;    // 第 i 个数字已经被占用，直接跳过
            used[i] = true;
            path.push_back(nums[i]);
            backtracking(nums, used);
            path.pop_back();
            used[i] = false;         }
    }
    vector<vector<int>> permute(vector<int>& nums) {
        vector<bool> used(nums.size(), false); //used数组表示每个数字是否被占用
        backtracking(nums, used);
        return result;
    }
};
```

2.3 N皇后问题

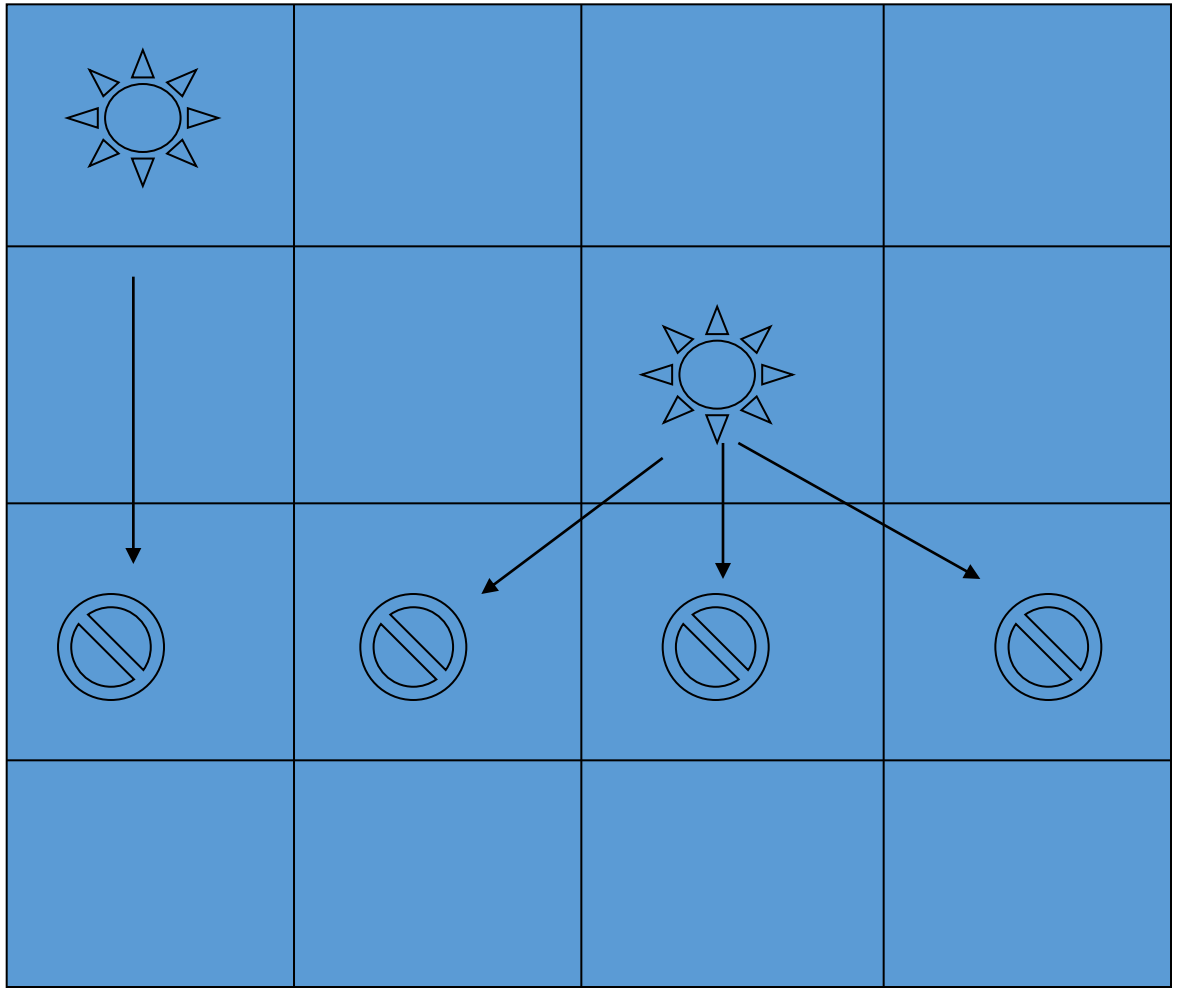
- 一个 n×n 的国际象棋棋盘上放置n个皇后，使其不能相互攻击，即任何两个皇后都不能处在棋盘的同一行、同一列、同一条斜线上，试问共有多少种摆法?



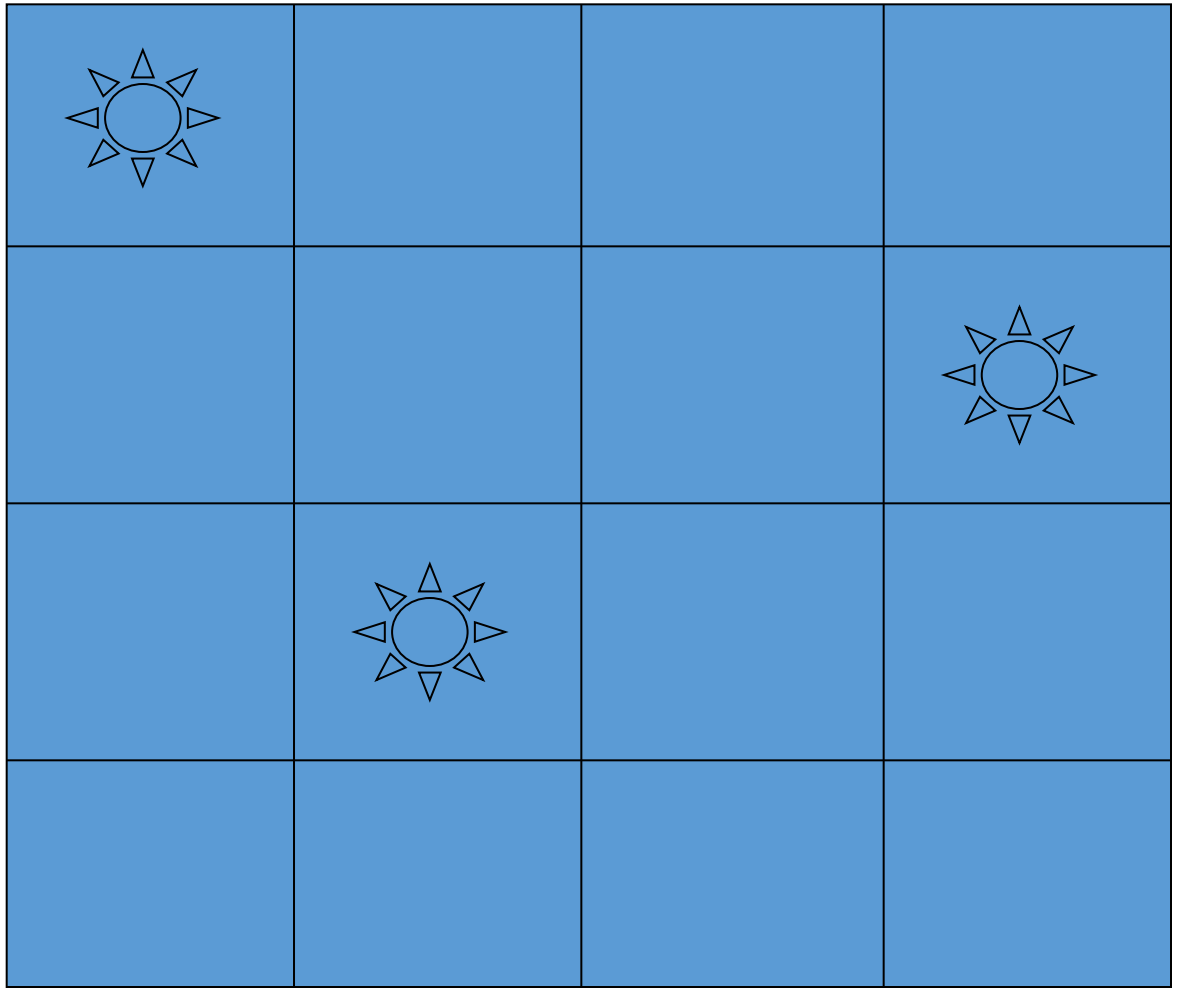
N(N=4)皇后问题



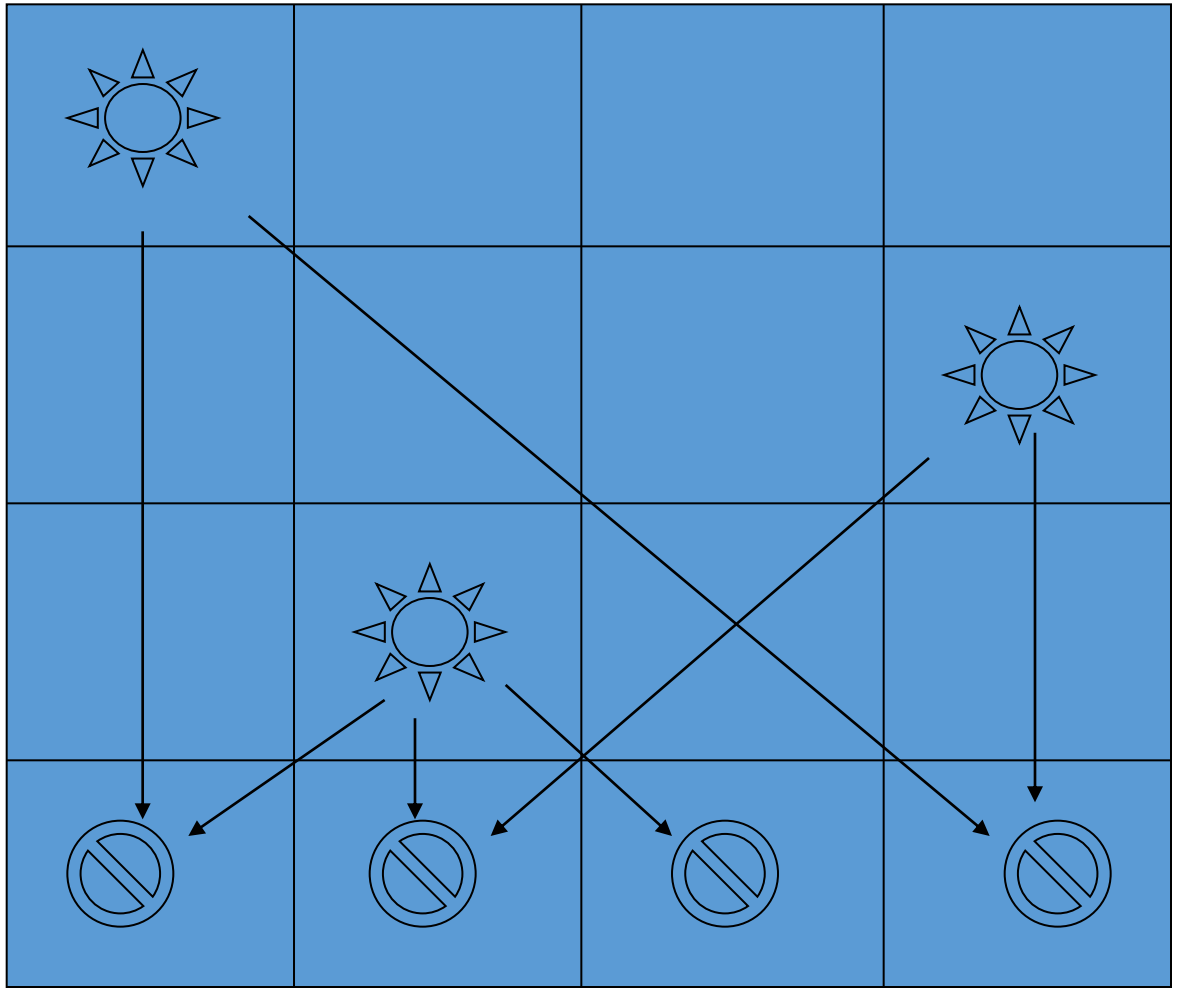
N(N=4)皇后问题



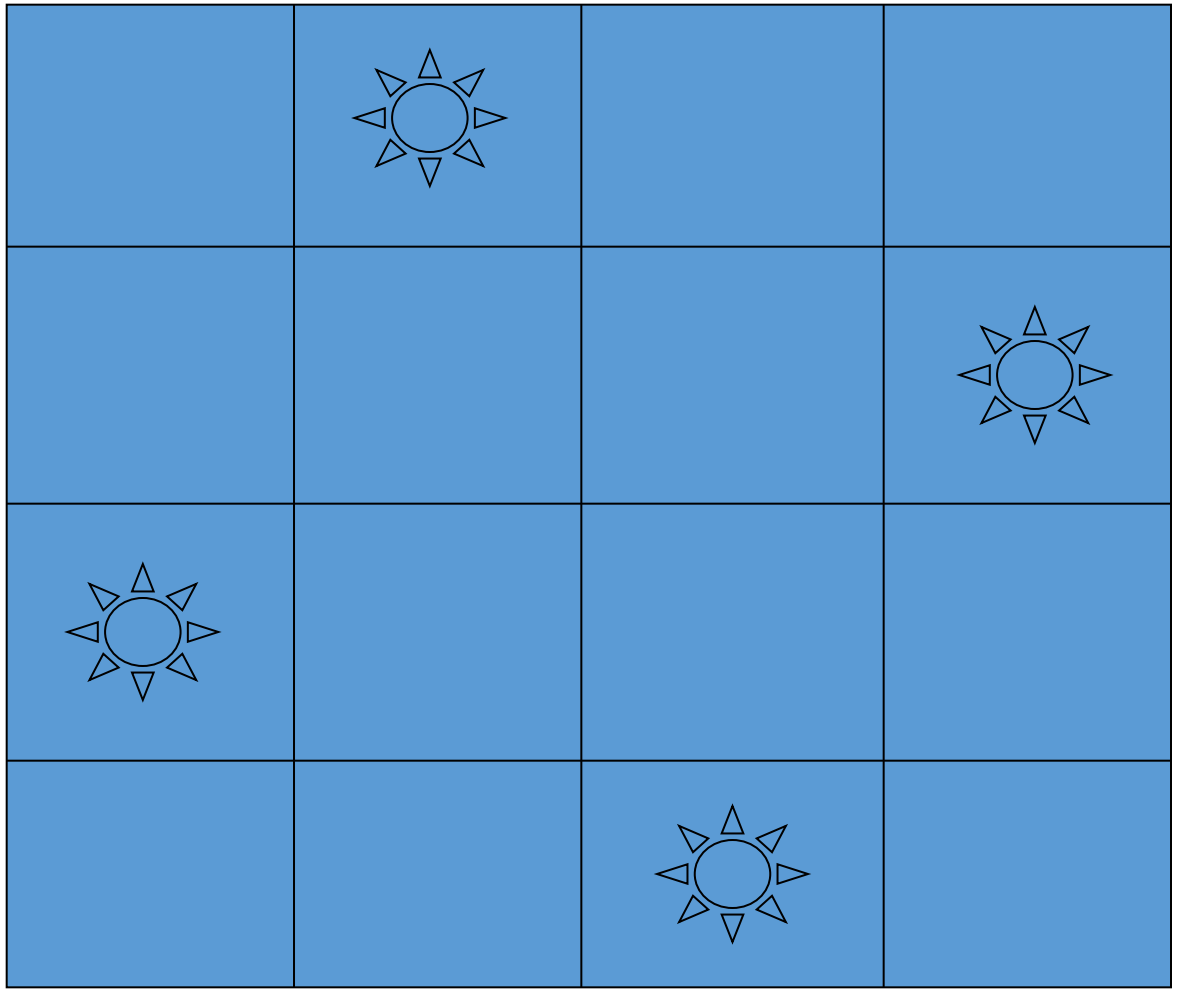
N(N=4)皇后问题



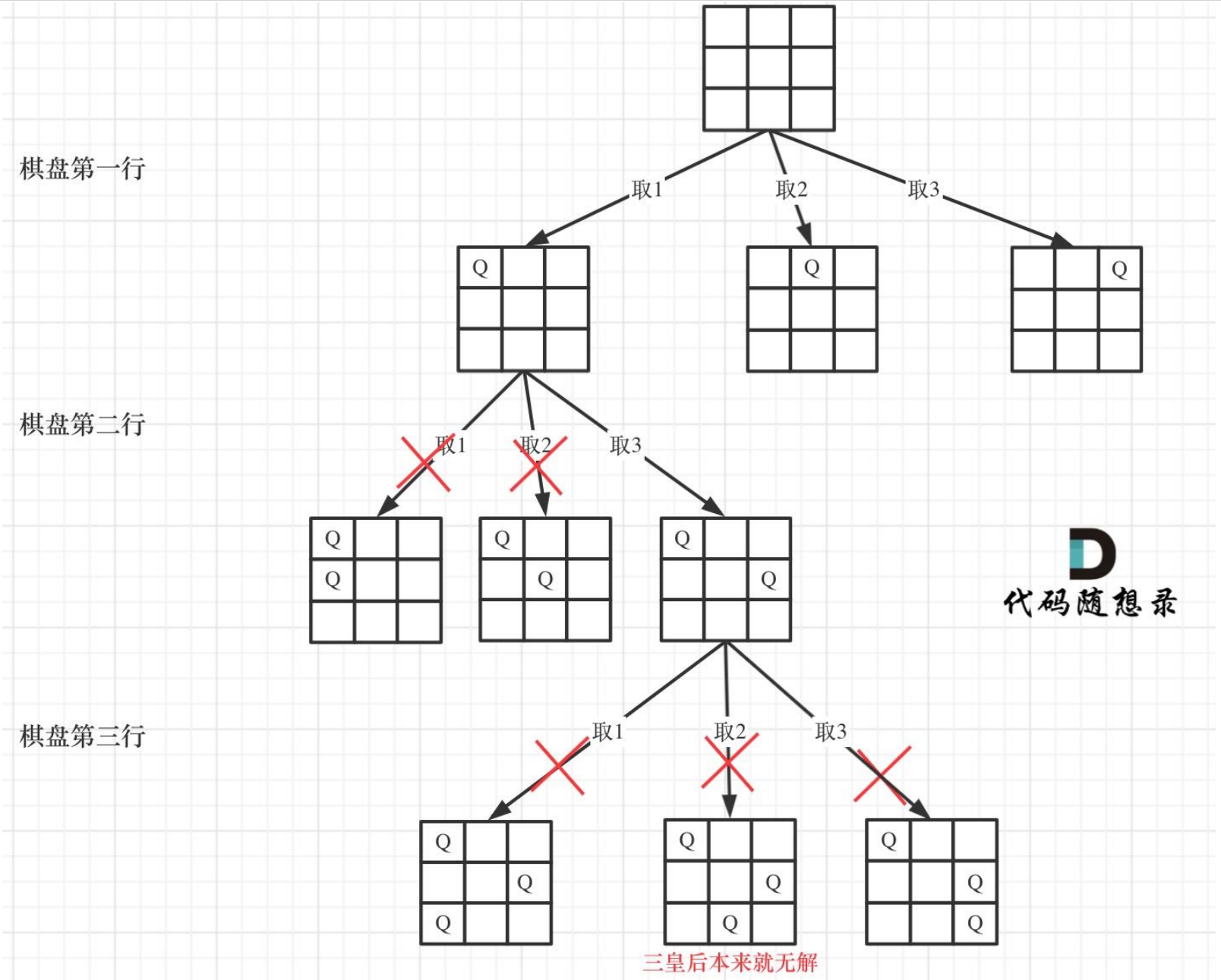
N(N=4) 皇后问题



N(N=4)皇后问题



3皇后问题的解空间树（部分）



N皇后问题算法分析-8后为例

设八个皇后为x_i，分别在第 i 行 (i=1，2，3，4……，8)；

- ✓ 问题的解状态：可以用$(1,x_1)$，$(2,x_2)$，……，$(8,x_8)$ 表示8个皇后的位置；

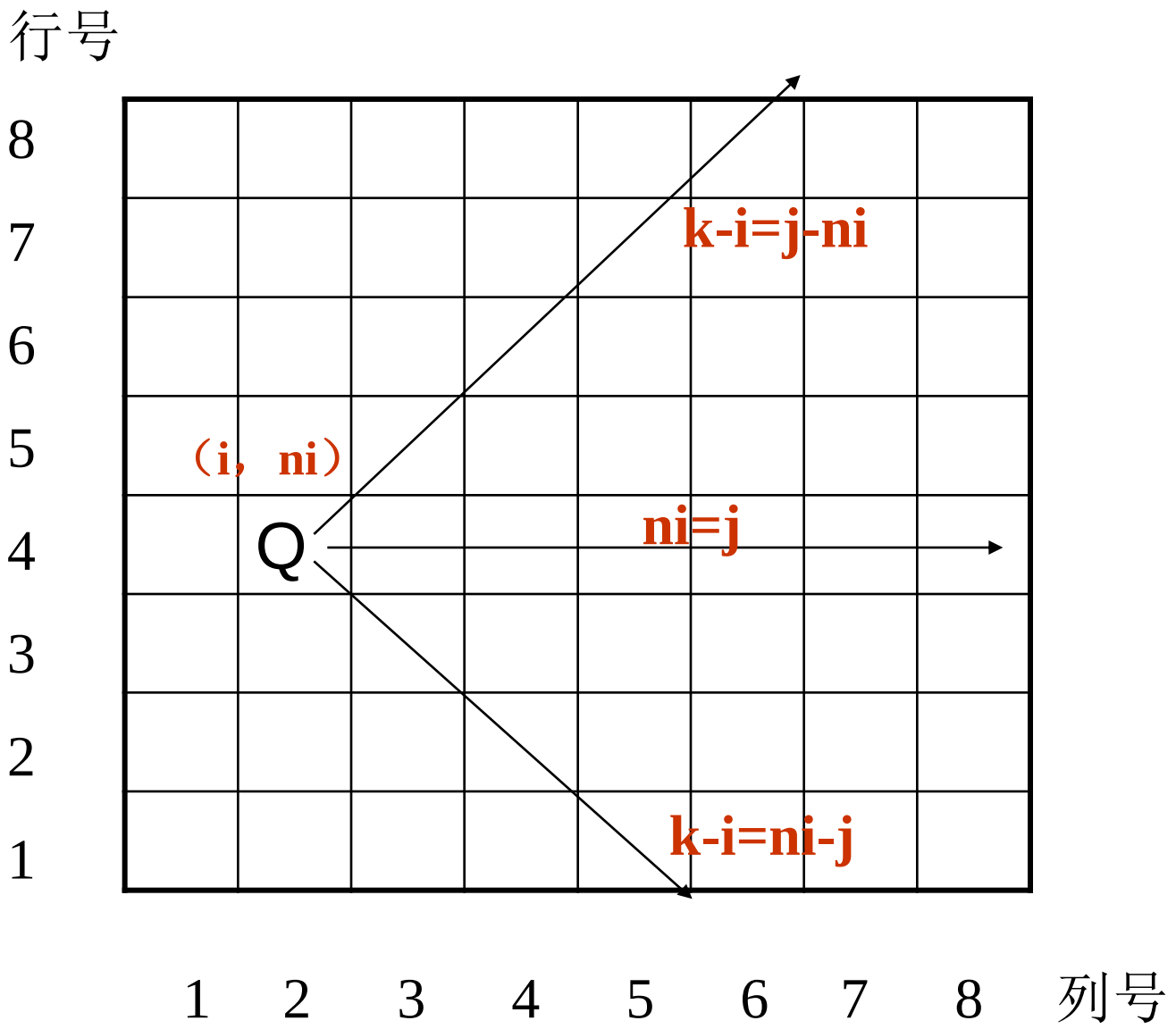
  由于行号固定，可简单记为：$(x_1,x_2,x_3,x_4,x_5,x_6,x_7,x_8)$；

- ✓ 问题的解空间：$(x_1,x_2,x_3,x_4,x_5,x_6,x_7,x_8)$，$1\le x_i\le 8$(i=1，2，3，4……，8)，共$8^8$个状态；

- ✓ 约束条件：八个$(1,x_1)$,$(2,x_2)$,$(3,x_3)$,$(4,x_4)$,$(5,x_5)$, $(6,x_6)$, $(7,x_7)$, $(8,x_8)$不在同一行、同一列和同一对角线上。

- 原问题即：在解空间中寻找符合约束条件的解状态。

- 按什么顺序去搜？
  - 目标是没有漏网之鱼，尽量速度快。


行号
8
7
6
5
4
3
2
1
k-i=j-ni
（i，ni）
Q
ni=j
k-i=ni-j
1 2 3 4 5 6 7 8
列号

N皇后问题约束条件

- 约束条件？
  - 不在同一列：$x_i \neq x_j$；
  - 不在同一主对角线上：$x_i - i \neq x_j - j$；
  - 不在同一负对角线上：$x_i + i \neq x_j + j$。
  - 冲突的情况可以整合改写为：
    - **abs(x_i - x_j)=abs(i-j)) or (x_i = x_j)**

通过8重循环模拟搜索空间中的8^8个状态

```
queen1( )
{
  int a[9];
  for (a[1]=1;a[1]<=8;a[1]++)
    for (a[2]=1;a[2]<=8;a[2]++)
      for (a[3]=1;a[3]<=8;a[3]++)
        for (a[4]=1;a[4]<=8;a[4]++)
          for (a[5]=1;a[5]<=8;a[5]++)
            for (a[6]=1;a[6]<=8;a[6]++)
              for(a[7]=1;a[7]<=8;a[7]++)
                for(a[8]=1;a[8]<=8;a[8]++)  {
                  if (check(a,8)==0) continue;
                  else
                    for(i=1;i<=8;i++)
                      print(a[i]);      }
}
```

用a[1]~a[8]存储x_1~x_8

```
check(a[],n)
{int i,j;
 for(i=2;i<=n;i++)
     for(j=1;j<=i-1;j++)
       if (a[i]=a[j]) or (abs(a[i]-a[j])=abs(i-j))
           return 0 ;
  return(1);
}
```

双重循环，任意两个皇后之间都必须检查。

注意：用固定的循环只能解固定规模的皇后问题（组合类问题）。只有用回溯法（递归）才能解任意规模的组合类问题。

N皇后问题的回溯法

```
//n：皇后个数
//x：当前解
//sum：当前已找到的可行方案数

void backtrack (int t)  {
    if (t>n) sum++;
    else
        for (int i=1;i<=n;i++) {
          x[t]=i;
          if (place(t))  backtrack(t+1);
        }
}

主程序调用：backtrack(1);
```

```
//判定两个皇后是否在同一斜线或在同一列上
bool place(int k)
{
  for (int j=1;j<k;j++)
      if ((abs(k-j)==abs(x[j]-x[k]))||(x[j]==x[k]))
        return false;
  return true;
}
```

函数Place(k) 测试待确定的第 k 个皇后是否和前面已确定的k-1个皇后在同一条斜角线或同一列上。

N皇后问题的回溯法（《代码随想录》）

```
class Solution {
private:
vector<vector<string>> result;
// n：棋盘大小， row：当前递归到棋盘的第几行了
void backtracking(int n, int row, vector<string>&
chessboard) {
    if (row == n) {
        result.push_back(chessboard);
        return;
    }
    for (int col = 0; col < n; col++) {
        if (isValid(row, col, chessboard, n)) {
            chessboard[row][col] = 'Q'; // 放置皇后
            backtracking(n, row + 1, chessboard);
            chessboard[row][col] = '.'; // 回溯，撤销皇后
        }
    }
}

主程序调用：backtracking(n, 0, chessboard);
```

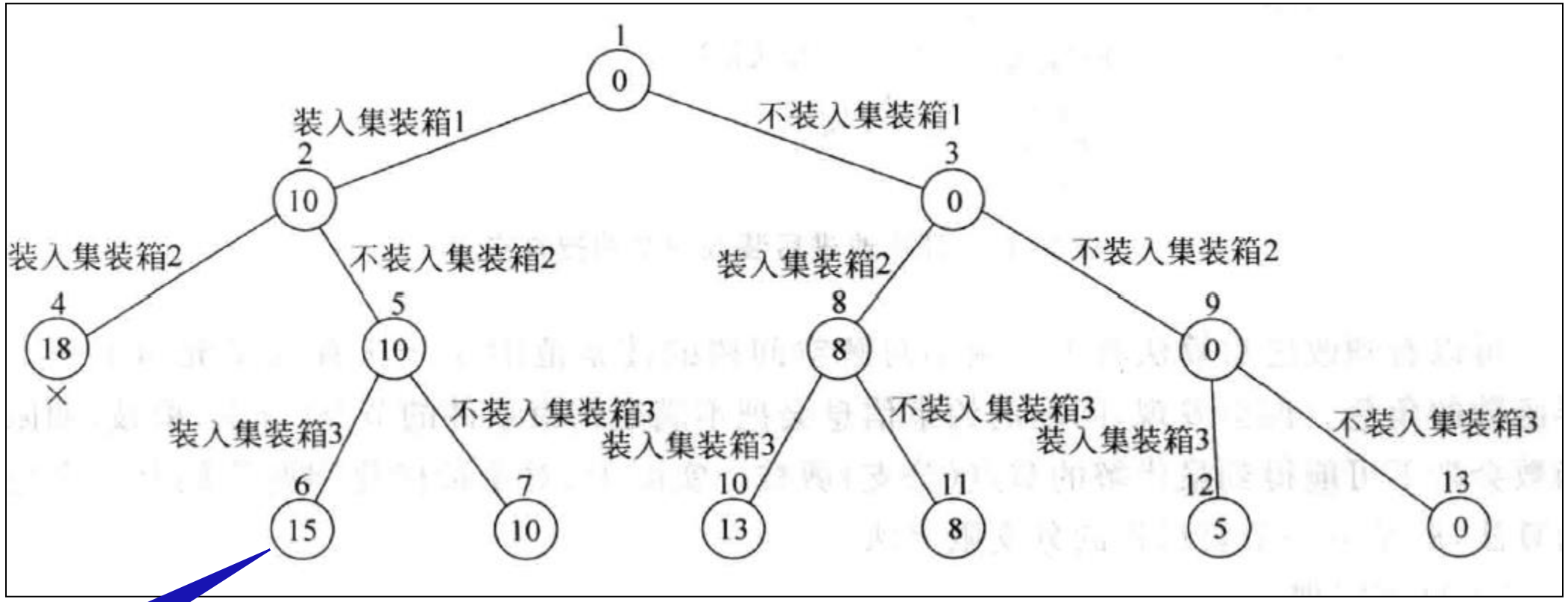
```
bool isValid(int row, int col, vector<string>& chessboard, int n)
{
    // 检查列
    for (int i = 0; i < row; i++) {
        if (chessboard[i][col] == 'Q') {
            return false;
        }
    }
    // 检查 45度角是否有皇后
    for (int i = row - 1, j = col - 1; i >=0 && j >= 0; i--, j--) {
        if (chessboard[i][j] == 'Q') {
            return false;
        }
    }
    // 检查 135度角是否有皇后
    for(int i = row - 1, j = col + 1; i >= 0 && j < n; i--, j++) {
        if (chessboard[i][j] == 'Q') {
            return false;
        }
    }
    return true;
}
```

2.4 装载问题

- 问题描述：
  - 共有 n 个集装箱要装在 2 艘载重量分别为 c1 和 c2 的轮船上，其中集装箱 i 的重量为 w_i，所有集装箱重量之和不超过c1+c2。
  - 是否有一个合理的装载方案可将这 n 个集装箱装上两艘轮船。如果有，找出一种装载方案。
- 如果一个装载问题有解，则采用下面的策略可以得到最优装载方案
  - 1、首先将第一艘轮船尽可能地装满。
  - 2、然后将剩余的集装箱装上第二艘轮船。
- 将第一艘轮船尽可能装满等价于如何选取全体集装箱的一个子集，使该子集中集装箱的重量之和最接近 c1。

装载问题（续）

- 解空间：子集树
- 约束条件：$\sum_{i=1}^{n} w_i x_i \leq c_1$
- 目标函数：$\min(c_1 - \sum_{i=1}^{n} w_i x_i)$ （使船的剩余重量达到最小）

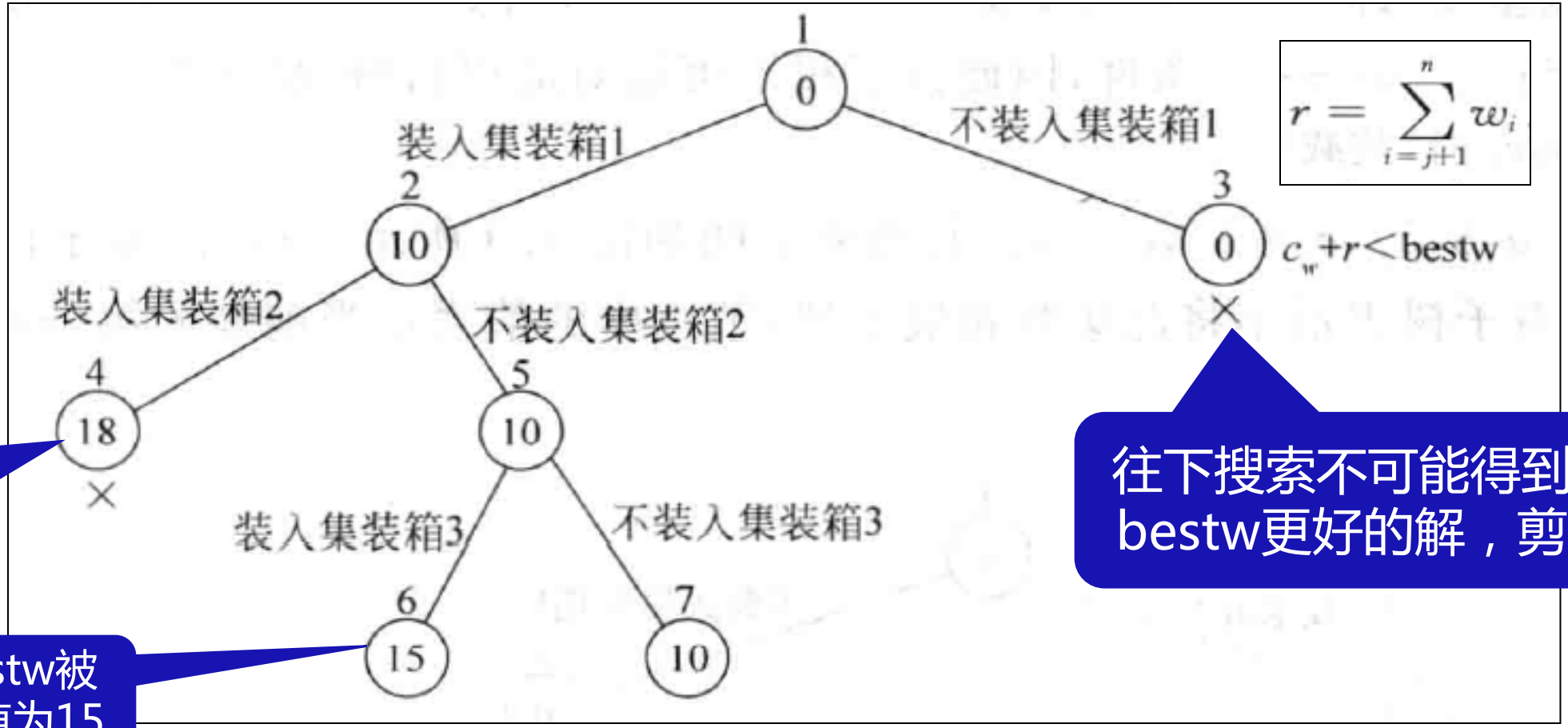


最优解

W=[10,8,5],c1=16,c2=8的搜索空间

装载问题（续）

- 限界函数：bestw > c_w +r，其中 c_w 为已经装入的重量，r 为剩余集装箱的重量和。
- 加入限界函数后，该问题的搜索空间如下：



往下搜索不满足约束条件，剪枝

往下搜索不可能得到比bestw更好的解，剪枝

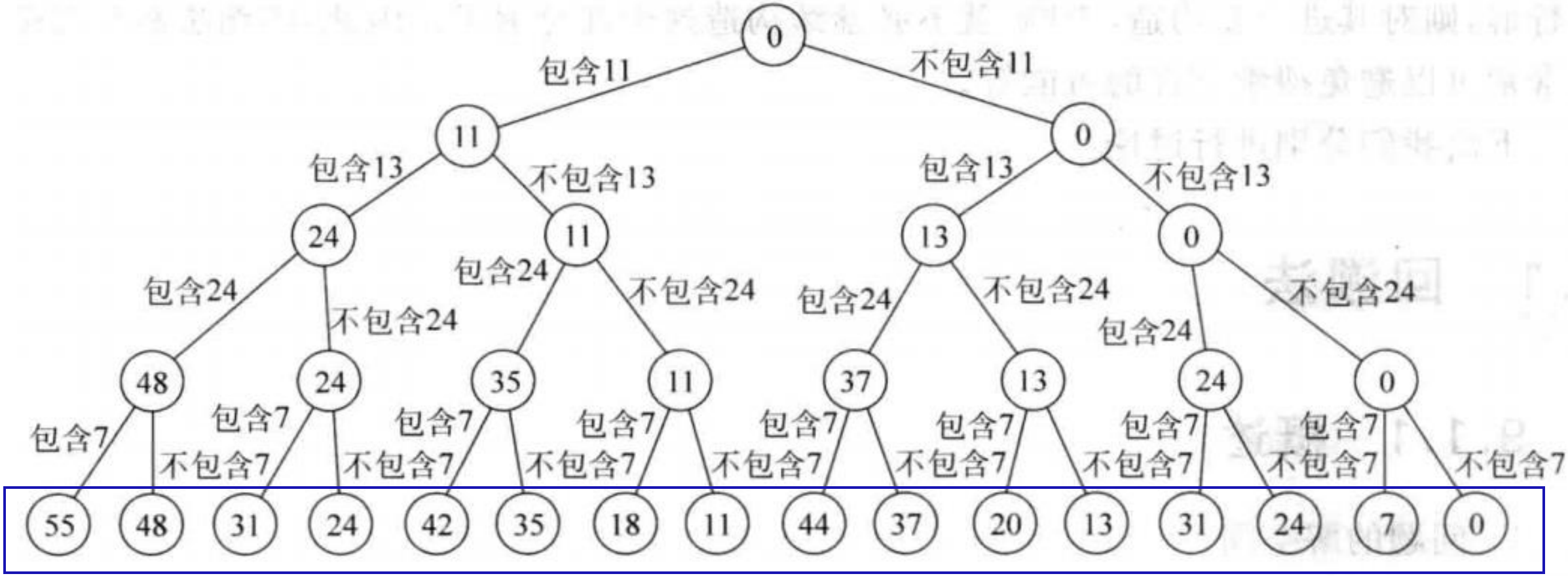
bestw被赋值为15

2.5 图的着色问题

- 引例：招聘问题
  - 某公司招聘新人，遇到这样的问题：很多应聘者想同时应聘多个职位，而应聘者不可能同时参加多个部门的面试，所以要想使每位应聘者在面试中都有机会和时间来应聘他们所选择的不同部门，就必须要给他们安排出合适的面试场次。为节省时间，公司希望安排的面试场次越少越好，以尽快结束招聘工作。
  - 例如：公司有5个部门招聘新人（分别用1，2，3，4，5表示），甲想应聘部门1，2，4；乙想应聘部门3，4，5；丙想应聘部门1，2，5。为了使他们在应聘中不发生冲突，公司至少要安排4场面试。
- 该问题可以采用图的着色来求解
  - 在本问题中，可以把每个部门作为图的一个顶点，如有应聘者同时应聘几个部门时，在这些部门之间相互连一条边，这样就得到一个图。
  - 对图的顶点进行着色，要求相邻的顶点必须着不同的颜色，同色顶点安排在同一场面试。这样能完成着色的最少颜色数即为需要安排的最少场次。
- 参见《图的着色问题.pptx》和实验2.3代码。

2.6 子集和问题

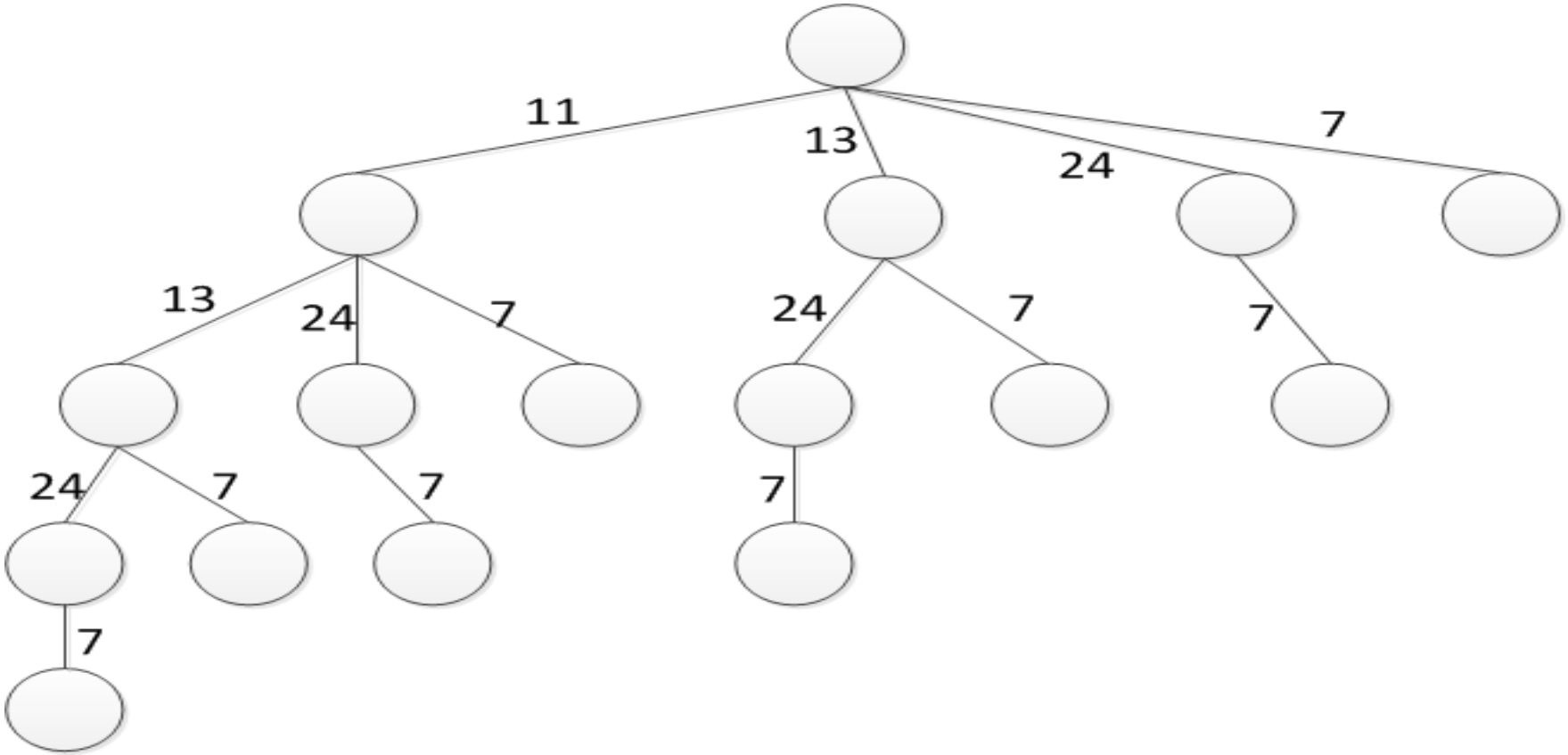
- 子集和问题的解空间树1（教材的例子: (11, 13, 24, 7)）



在该子集树中，只有“叶节点”是可能解，叶节点代表了4个数字的16种组合情况

子集和问题（续）

- 子集和问题的解空间树2（参见《代码随想录》）



整个树的节点总数为 16

在该子集树中，每条边代表选择某个数字，树中所有节点都是可能解，代表了4个数字的16种组合情况（回溯该子集树的代码见下页）

```
class Solution {
private:
    vector<vector<int>> result;
    vector<int> path;
    void backtracking(vector<int>& nums, int startIndex) {
        result.push_back(path); // 收集子集（中间节点也是子集）
        if (startIndex >= nums.size()) {// 终止条件可以不加
            return;       }
        for (int i = startIndex; i < nums.size(); i++) {
            path.push_back( nums[i] );
            backtracking(nums, i + 1 ); //从本层的下一个数字继续递归搜索
            path.pop_back();          }
    }
public:
    vector<vector<int>> subsets(vector<int>& nums) {
        result.clear();
        path.clear();
        backtracking(nums, 0);
        return result;     }     };
```

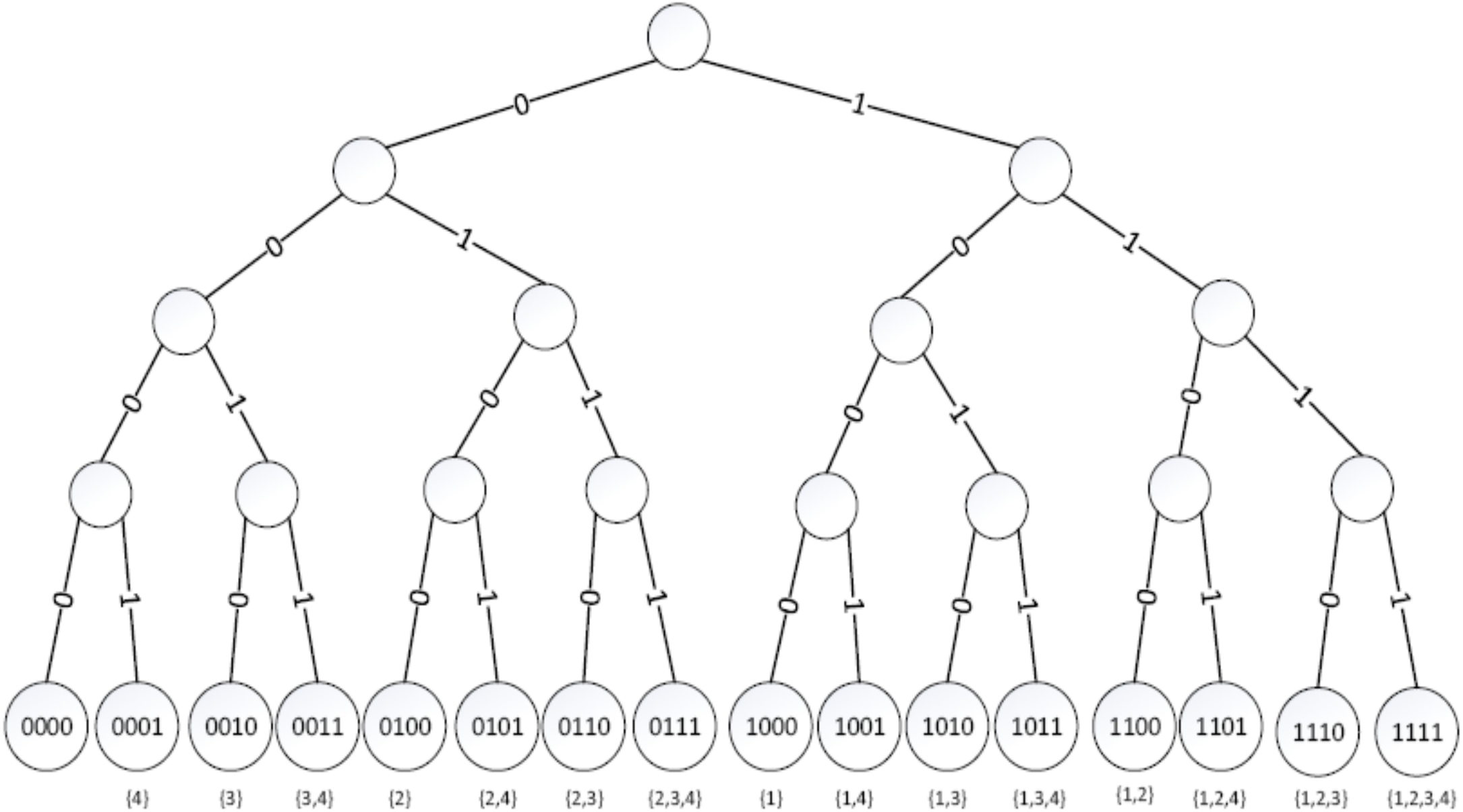
《代码随想录》中的代码

三、分支限界法

- 3.1 分支限界法概述
- 3.2 分支限界法和回溯法的区别
- 3.3 分支限界法的关键问题
- 3.4 两种分支限界法
- 3.5 分支限界法的一般步骤

3.1 分支限界法概述

- 分支限界法=分支（ Branch ）+限界（Bound）
  - 分支：搜索解空间树时按照最佳优先策略形成“跳跃式”搜索，而不是标准的深度优先或广度优先搜索。
  - 限界：在搜索时按照“限界函数”（启发函数）的值确定下一步扩展的节点（最有希望达到最优值的节点），从而形成跳跃式搜索。
- 限界函数（bound method）
  - 限界函数的值 = 当前节点已经获得的值+从该节点往下能获得的理论上的最优值（一般利用贪心法计算）。
  - 最大值问题的限界函数计算理论的上界；最小值问题的限界函数计算理论的下界。
- 分支限界法的关键数据结构：优先队列（堆）。
- 分支限界法适用于求解最优化问题。


0000
0001
0010
0011
0100
0101
0110
0111
1000
1001
1010
1011
1100
1101
1110
1111
{4}
{3}
{3,4}
{2}
{2,4}
{2,3}
{2,3,4}
{1}
{1,4}
{1,3}
{1,3,4}
{1,2}
{1,2,4}
{1,2,3}
{1,2,3,4}

3.2 分支限界法和回溯法的区别

- 1、对解空间树的搜索方式不同
  - 回溯法采用深度优先搜索，分支限界法采用最佳优先搜索。
  - 回溯法的搜索过程可描述为“先从一条路往前走，能进则进，不能进则退回来，换一条路再试”，而分支限界法可描述为“先选择一条最有希望的路往前走，每走一步都要看看是否出现更有希望的路，如果有则选择更有希望的路往前走”。
- 2、算法的非递归形式实现不同
  - 回溯法采用栈（后进先出）来实现深度优先搜索。
  - 分支限界法通常采用优先队列（堆）来实现最佳优先搜索。
- 3、求解目标不同
  - 回溯法的目标通常是找出解空间中满足约束条件的一个解（或所有解）。
  - 分支限界法的目标通常是找出满足约束条件的一个最优解。

分支限界法和回溯法的区别（续）

- 4、对当前扩展节点使用的扩展方式不同
  - 在分支限界法中，每一个活节点只有一次机会成为扩展节点。活节点一旦成为扩展节点，就一次性产生其所有儿子节点。在这些儿子节点中，那些导致不可行解或导致非最优解的儿子节点被舍弃，其余儿子节点被加人到活节点表中。此后，从活节点表中取下一个节点成为当前扩展节点，并重复上述节点扩展过程。这个过程一直持续到找到所求的解或活节点表为空时为止。
  - 回溯法是每次只扩展活节点的一个儿子节点。

3.3 分支限界法的关键问题

- 1、如何确定合适的限界函数
  - 分支限界法在遍历过程中根据限界函数估算某节点对应的目标函数的可能取值。
  - 好的限界函数不仅要计算简单，还要保证最优解在搜索空间中，更重要的是能在搜索的早期对超出目标函数界的节点进行丢弃，减少搜索空间，从而尽快找到问题的最优解。
- 2、如何组织待处理节点
  - 为了能在待处理节点中选取使目标函数取得极值（极大或极小）的节点，可以采用“堆”存储待处理节点。
  - 待处理节点的关键字（堆的排序依据）为限界函数的值。
  - 对于最大化问题采用最大堆，对于最小化问题采用最小堆。

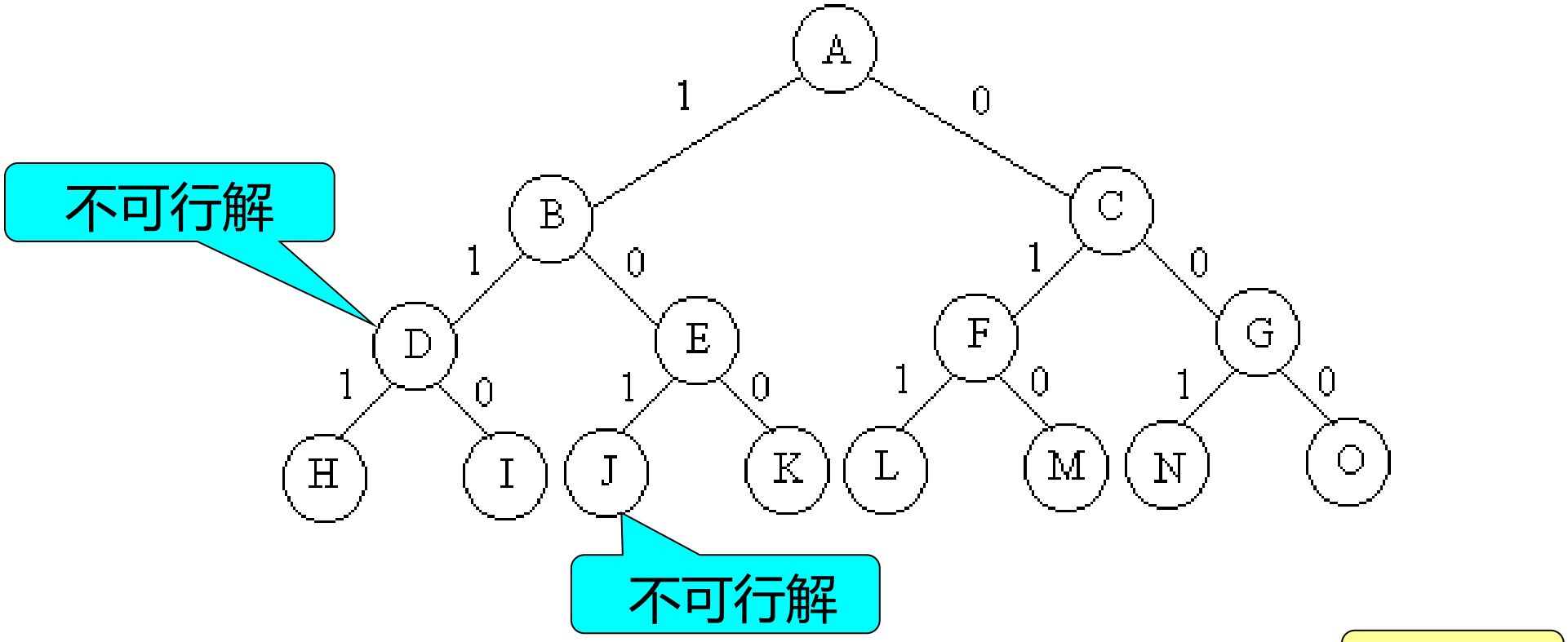
分支限界法的关键问题（续）

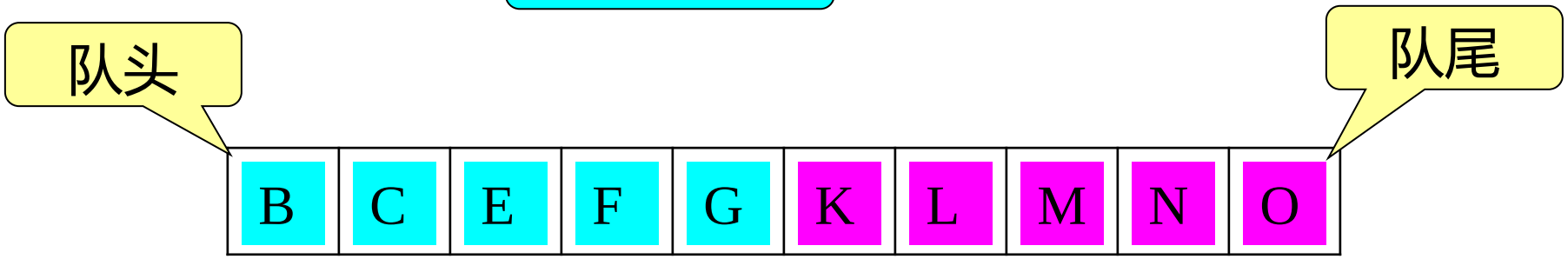
- 3、如何记录最优解中的各个分量
  - 分支限界法对问题的解空间树中节点的处理是跳跃式的，它会从待处理节点中选择一个最可能导致最优解的节点来处理，这个节点可能是刚刚处理过节点的孩子节点，也可能是兄弟节点，也可能是上层节点。
  - 因此要记录最优解的分量，不能用回溯法中的全局解向量，因为回溯采用的是深度优先策略，任何时刻只需要保存从根到正在处理节点的“一条”分支的路径；而分支限界法需要保存从根下来到所有待处理的节点的“多条”分支的路径。
  - 解决这个问题的一个方法是在每个节点中存储从根节点到该节点的路径，这样在找到目标节点后，可以直接输出最优解的解向量。
    - 为此，在定义节点的数据结构时，除包括该节点对应的限界函数的值等信息外，还应包括从根到该节点的路径，即部分解（x_1、x_2、…、x_i）。

3.4 两种分支限界法

- 按照存储活动结点数据结构的不同，可以将分支限界法分为：
  - 1、队列式分支限界法：用队列存储活动结点，按照先进先出（FIFO）策略选取下一个结点为扩展结点。
  - 2、优先队列式分支限界法：用优先队列（堆）存储活动结点，按照优先级选取优先级最高的结点成为当前扩展结点，即最佳优先搜索策略。
- 队列式分支限界法与广度优先算法极为相似，唯一的不同是队列式分支限界法不搜索以不可行节点为根的子树。

队列式分支限界法的例子（复习）





活结点队列

3.5 分支限界法的一般步骤

- 优先队列式分支限界法的特点：
  - 结合了深度优先和广度优先的优点。
  - 该过程每次都是选择到目前为止最可能产生最优解的分支进行扩展，具有全局观点。
  - 与回溯法相比，分支限界法可以根据限界函数不断调整搜索方向，选择最有可能取得最优解的子树优先进搜索，从而尽快得到问题的解。
- 优先队列式分支限界法的一般步骤：
  - 1、使用限界函数构造一个堆（最大堆或最小堆），并首先将根节点加入到堆中。
  - 2、考察该堆的根元素是否是目标节点，如果是则停止；否则转向步骤 3。
  - 3、从该堆中删除根元素，并扩展该元素，把这个元素的后代添加到堆中。
  - 4、如果该堆是空的，那么失败，否则转向步骤 2。

四、常见的分支限界算法

- 4.1 0-1背包问题
- 4.2 任务分配问题
- 4.3 旅行商问题
- 4.4 多段图最短路径
- 4.5 单源最短路径

4.1 0/1背包问题

- 假设有 4 个物品，其重量分别为(4, 7, 5, 3)，价值分别为(40, 42, 25, 12)，背包容量W=10。
- 将给定物品按单位重量价值从大到小排序，结果如下：

物品	重量(w)	价值(v)	价值/重量(v/w)
1	4	40	10
2	7	42	6
3	5	25	5
4	3	12	4

0/1背包问题（续）

分析：问题的解可表示为 n 元向量 $\{x_1, x_2, \ldots x_n\}$，$x_i \in \{0,1\}$，可用子集树表示解空间，在树中做广度优先搜索。

约束条件：$\sum_{i=1}^{n} w_i x_i \le W$　　目标函数：$V = \max \sum_{i=1}^{n} v_i x_i$

下界：V_{db} =40　(1, 0, 0, 0)——这个是可行解，可用于剪枝

上界：$V_{ub} = 0+(W-0)\times(v_1/w_1) = 0+(10-0)\times 10 = 100$

目标函数的界：[40, 100]，界通常根据目标函数设定

上界函数：

$$ub = v + (W - w) \times (v_{i+1} / w_{i+1})$$

v：前i个物品获得的价值

$(W - w) \times (v_{i+1} / w_{i+1})$：剩余容量全部装入物品$i+1$，最多能够获得的价值（注意物品已按单位价值排序）

ub 随着程序运行会不断变化

最大化问题关键要确定上界函数（理论上最优值）

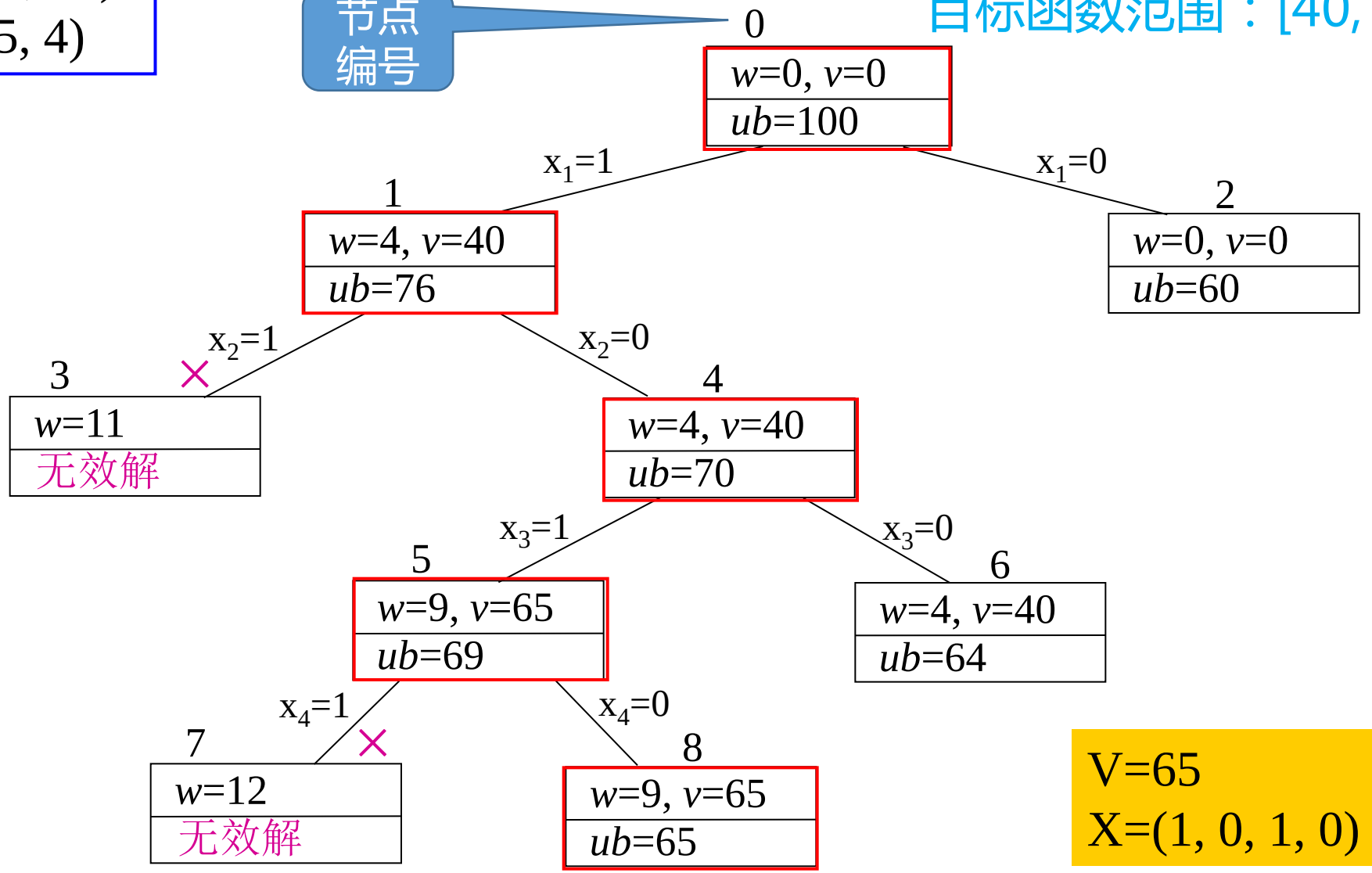
0/1背包问题（续）

w_i=(4, 7, 5, 3)
v_i= (40, 42, 25, 12)
v_i/w_i=(10, 6, 5, 4)

$$ub = v + (W - w) \times (v_{i+1}/w_{i+1})$$

节点编号

目标函数范围：[40, 100]

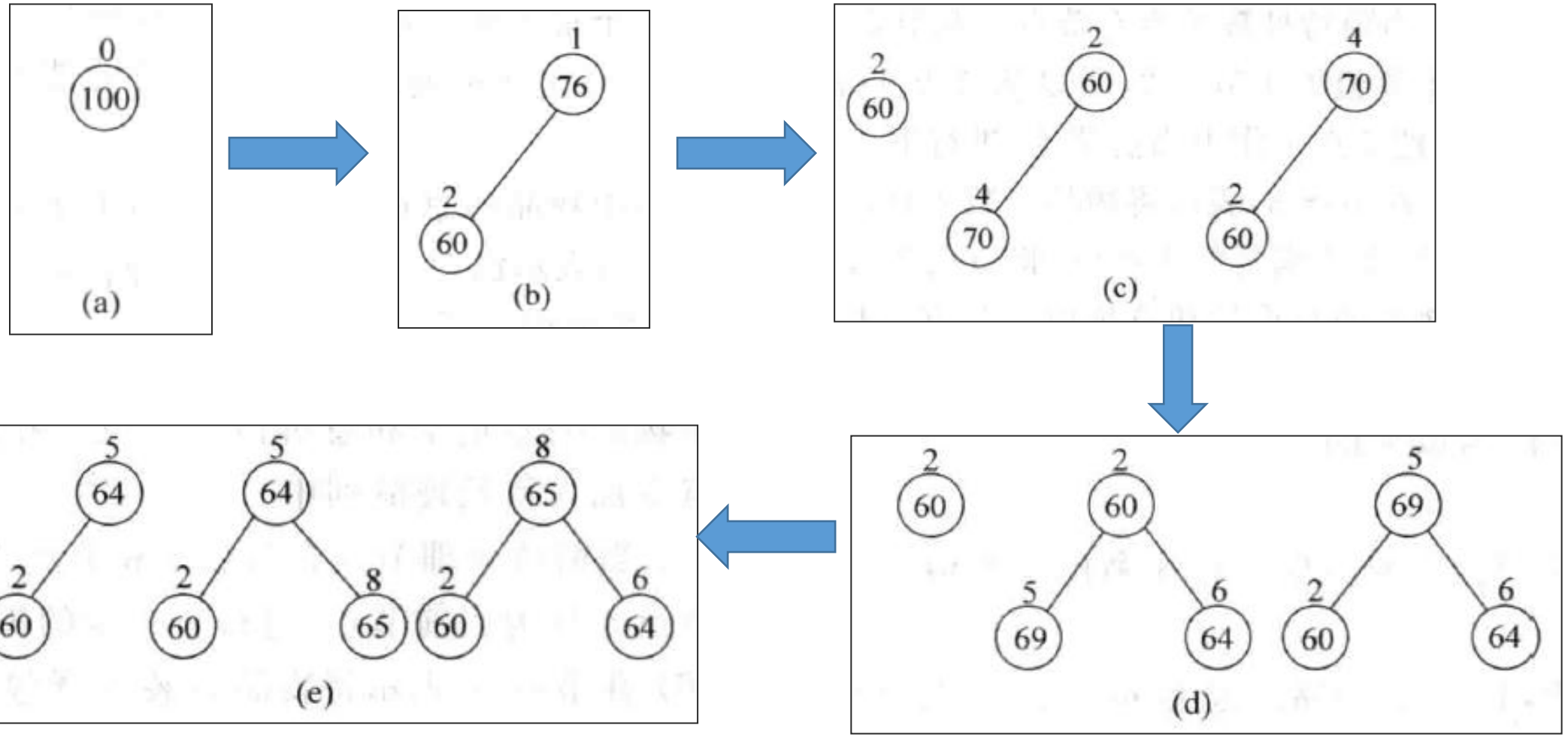


需要掌握ub的计算方法

V=65
X=(1, 0, 1, 0)

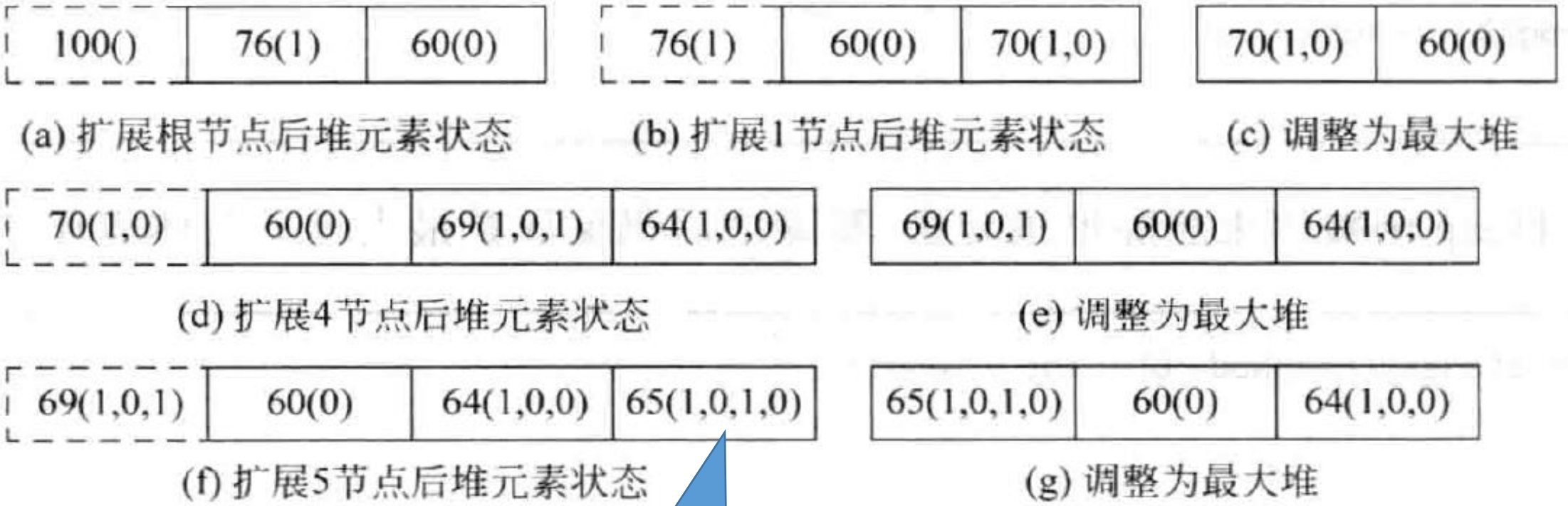
0/1背包问题（续）

- 0-1背包问题是求最大值问题，用最大堆来实现，把当前背包中物品总价值的上界（ub）作为堆的关键字。
- 其最大堆的构建过程如下（必须熟练掌握）：



0/1背包问题（续）

- 此外，由于是跳跃式搜索，需要在最大堆元素（节点）中存储每个节点的路径信息：



0/1背包问题核心代码

```
struct HeapNode{
        int w;                          //当前重量
        int v;                          //当前价值
        int level;                      //层级
        int bound;                      //上界
        int route[N];                   //路径,代表了解向量的各个分量
    };
```

活节点结构

```
while(heapLength > 0)
{
    HeapNode temp;
    temp = heap[1];
    level = temp.level;
    DeleteHeap(heap, heapLength);
    if(level == n&&temp.v >= MaxUpBound(heap, heapLength))
    {
        for (i = 1 ; i <= n ; i ++)
            输出 temp.route[i];
        输出 temp.v;
        return ;
    }
    else
```

堆顶元素即为bound最大的元素

删除堆顶元素（只扩展一次）

当前节点的v必须比活结点表中所有节点的上界都大

```
else
{
    for(j = 0;j <= 1;j++)
    {
        if(temp.w + j * w[level + 1]<= W)
        {
            HeapNode node;
            node.w = temp.w + j * w[level + 1];
            node.v = temp.v + j * v[level + 1];
```

约束条件

扩展temp的后代

0/1背包问题核心代码（续）

```
        if(level <= n - 1)
        {
            node.bound = node.v + (W - node.w) * (v[level + 1]/w[level + 1]);
        }
        else
        {
            node.bound = node.v;
        }
        for(i = 1;i <= level;i++)
            node.route[i] = temp.route[i];
        node.route[level + 1] = j;
        node.level = level + 1;
        InsertHeap(heap,node,heapLength);
    }
}
```

计算新节点的上界

计算新节点的路径

将新节点加入最大堆

4.2 任务分配问题

- 任务分配问题：
  - 要求把 n 项任务分配给 n 个人，每个人完成每项任务的成本不同，要求分配总成本最小的最优分配方案。
  - 约束函数：每个任务只能分配一个一个人。
  - 目标函数：min(总的任务成本)

	任务 1	任务 2	任务 3	任务 4	
$C=$	9	2	7	8	人员 a
	6	4	3	7	人员 b
	5	8	1	8	人员 c
	7	6	9	4	人员 d

成本矩阵的例子

任务分配问题

- 上界
  - 利用贪心算法，从第一行开始，每行选择一个成本最小的不重复任务。
  - 对于上例：ub = 2+3+5+4 = 14（这是一个有效的解）
- 下界
  - 直接将每行的最小值相加，得到下界（该下界并不一定是合法的分配方案，只是理论上的最小值）。
  - 对于上例：lb = 2+3+1+4 = 10
- 下界函数：如果已对人员 1 到 i 分配了任务，得到了成本 v，则

$$\text{lb} = v + \sum_{k=i+1}^{n} \text{第 } k \text{ 行的最小值}$$

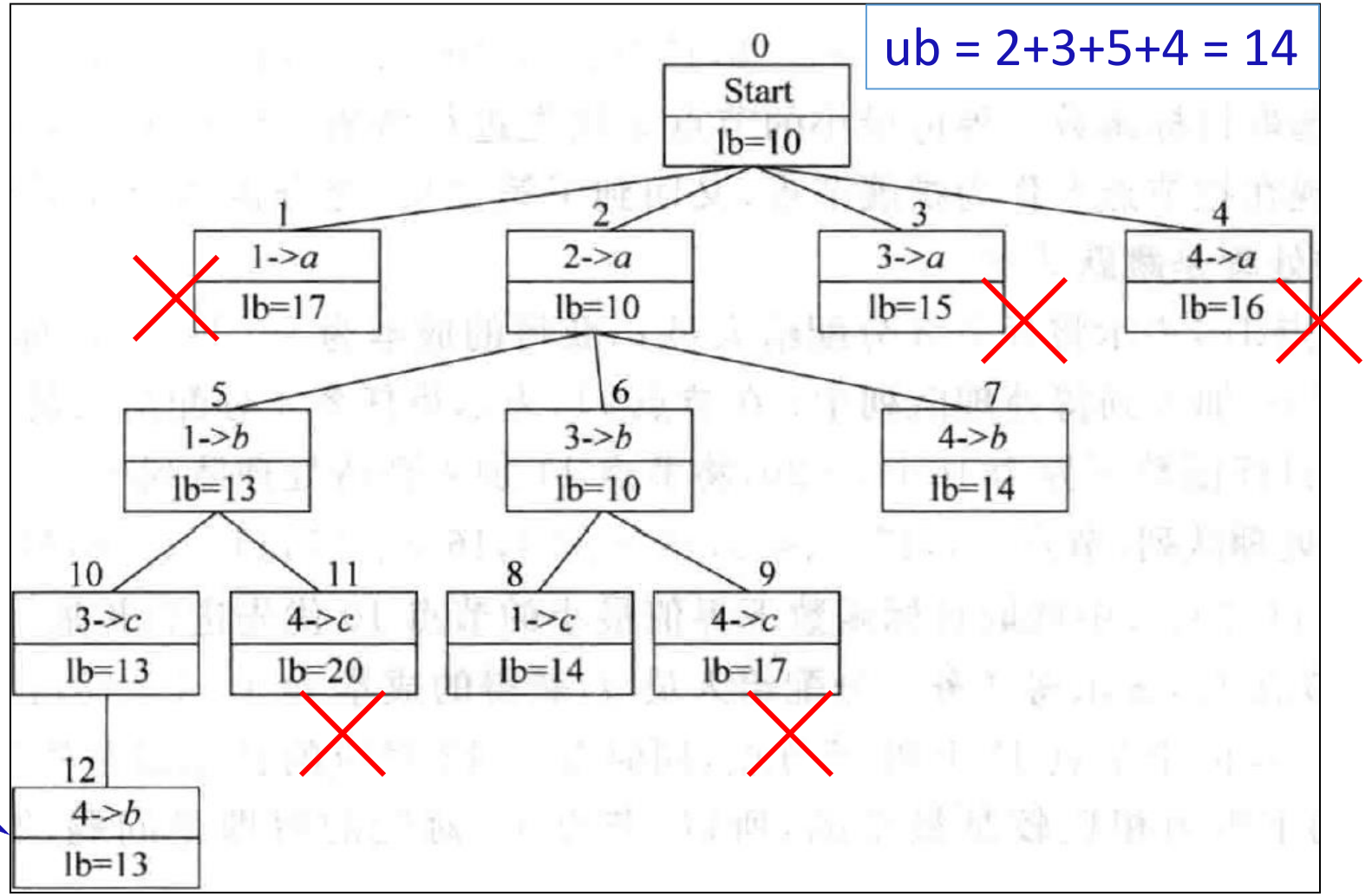
前 i 个人的最小成本

后面的人理论上能达到的最小成本

任务分配问题（续）

- 搜索策略：该问题是最小化问题，应该首先把具有最小下界（对应目标函数）的节点作为最有希望的节点进行扩展。

例子的搜索空间树



可行解，并且比活结点表中所有节点的下界都小

任务分配问题（续）

```
struct HeapNode{
        int v;                          //当前成本
        int level;                      //层级
        int bound;                      //上界
        int route[N+1];                 //路径,代表了解向量的各个分量
    };
```

```
void Assign(int C[N][N],int upBound)
{
    HeapNode rootNode;                          //构
    rootNode.v = 0;
    rootNode.bound = LineMinSum(C,0);
    for(i = 0;i < N;i++)
        rootNode.route[i] = 0;
    rootNode.level = 0;
    InsertHeap(heap,rootNode,heapLength);
    while(heapLength > 0)
```

```
while(heapLength > 0)
{
    HeapNode temp;
    temp = heap[1];                         /
    level = temp.level;
    DeleteHeap(heap,heapLength);
    //判堆顶是否是最终解
    if(level == N&&temp.v <= MinDownBound(heap,heapLength))
    {
        for (i = 1 ; i <= N ; i ++)
            输出 temp.route[i];  //打印输出问题的解
        输出 temp.v;
        return ;
    }
```

可行解，并且比活结点表中所有节点的下界都小

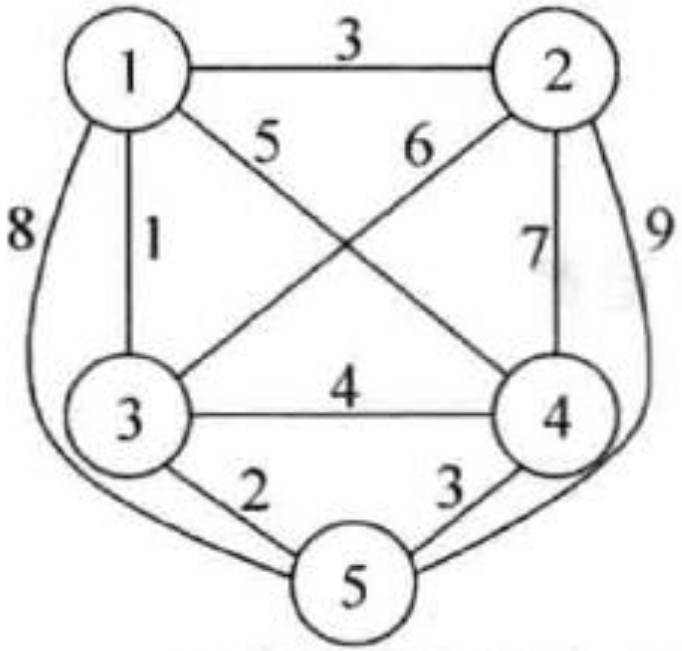
任务分配问题（续）

```
for(j = 1;j <= N;j++)
{                                    //只有满足约束条件的孩子节点才入堆
    if(!Contain(temp.route,level,j)&&(temp.v + C[level][j - 1]<= upBound))
    {
        HeapNode node;
        node.v = temp.v + C[level][j - 1];
        if(level <= N - 1)
        {
            node.bound = node.v + LineMinSum(C,level + 1);
        }
        else
        {
            node.bound = node.v;
        }
        for(i = 1;i <= level;i++)
            node.route[i] = temp.route[i];
        node.route[level + 1] = j;
        node.level = level + 1;
        InsertHeap(heap,node,heapLength);
    }
}
```

4.3 旅行商问题

- 一个旅行商问题的例子如下。
- 上界：采用贪心法求解。
  - 例子的上界为 1+2+3+7+3 =16

这是一个可行解，可以用于剪枝



(a) 一个无向图

$$C=\begin{pmatrix} \infty & 3 & 1 & 5 & 8 \\ 3 & \infty & 6 & 7 & 9 \\ 1 & 6 & \infty & 4 & 2 \\ 5 & 7 & 4 & \infty & 3 \\ 8 & 9 & 2 & 3 & \infty \end{pmatrix}$$

(b) 无向图的代价矩阵

旅行商问题（续）

- 下界：
  - 对每个城市 i，求出从城市 i 到最近的两个城市的距离和 s_i，计算出这 n 个数字的和 s，并把结果除以2，就得到一个合理的下界。
  - 这样考虑的原因是需要通过一条路径进人某城市，通过另外一条不同的路径离开这个城市，每一个城市都是如此，但距离多加了一倍，所以要把结果除以 2。
  - 注意：这种做法不保证是一个合法的选择，可能没有构成哈密顿回路，它仅仅给出一个参考下界（理论上最优值）。
  - 上例的下界为：

$$\text{lb} = ((1+3)+(3+6)+(1+2)+(3+4)+(2+3))/2 = 14$$

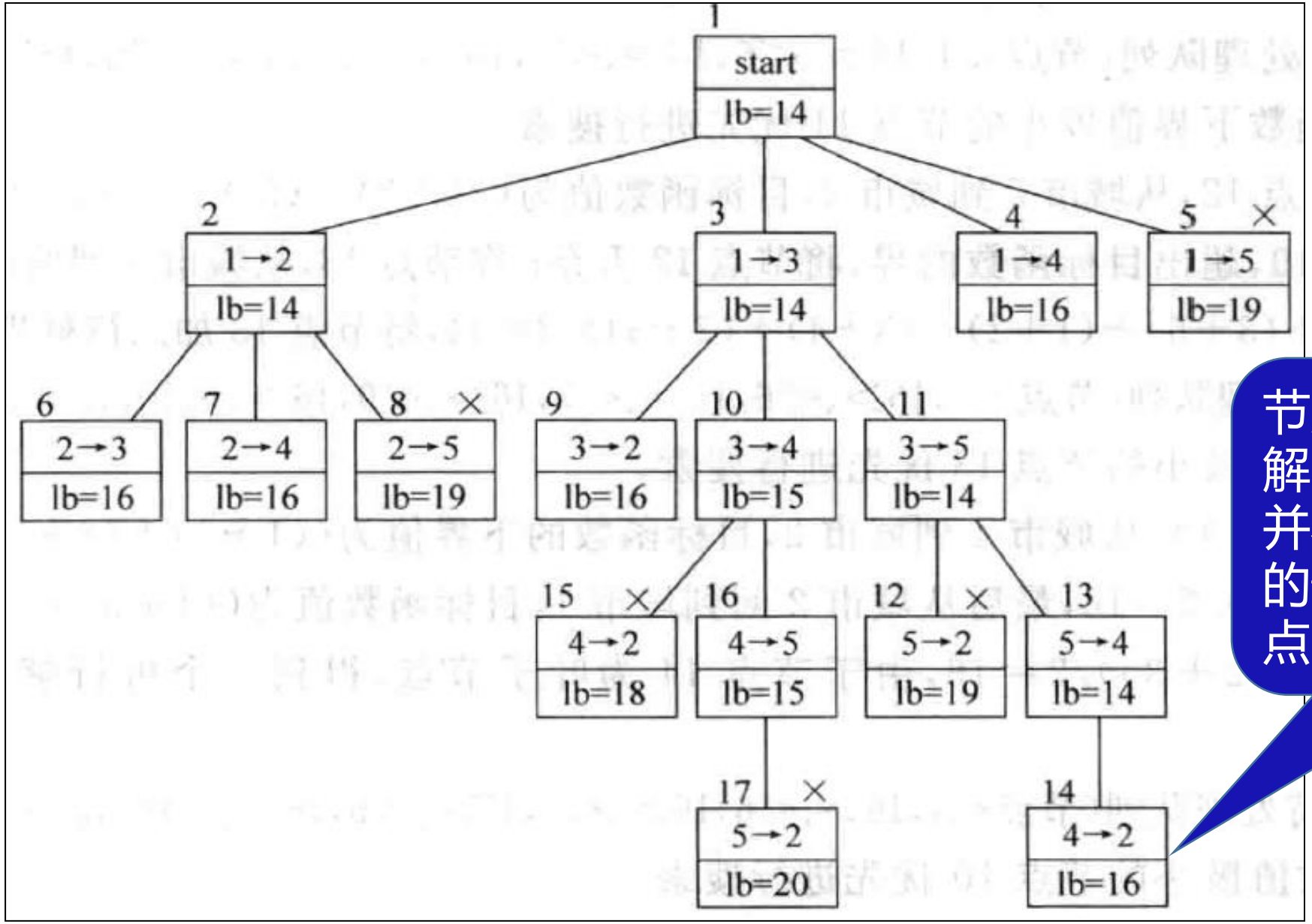
- 下界函数为：

$$\text{lb} = \sum s_i + \sum s_j$$

$s_i = i$ 行最小的两个元素之和，表示城市 i 不和已确定的边相连。

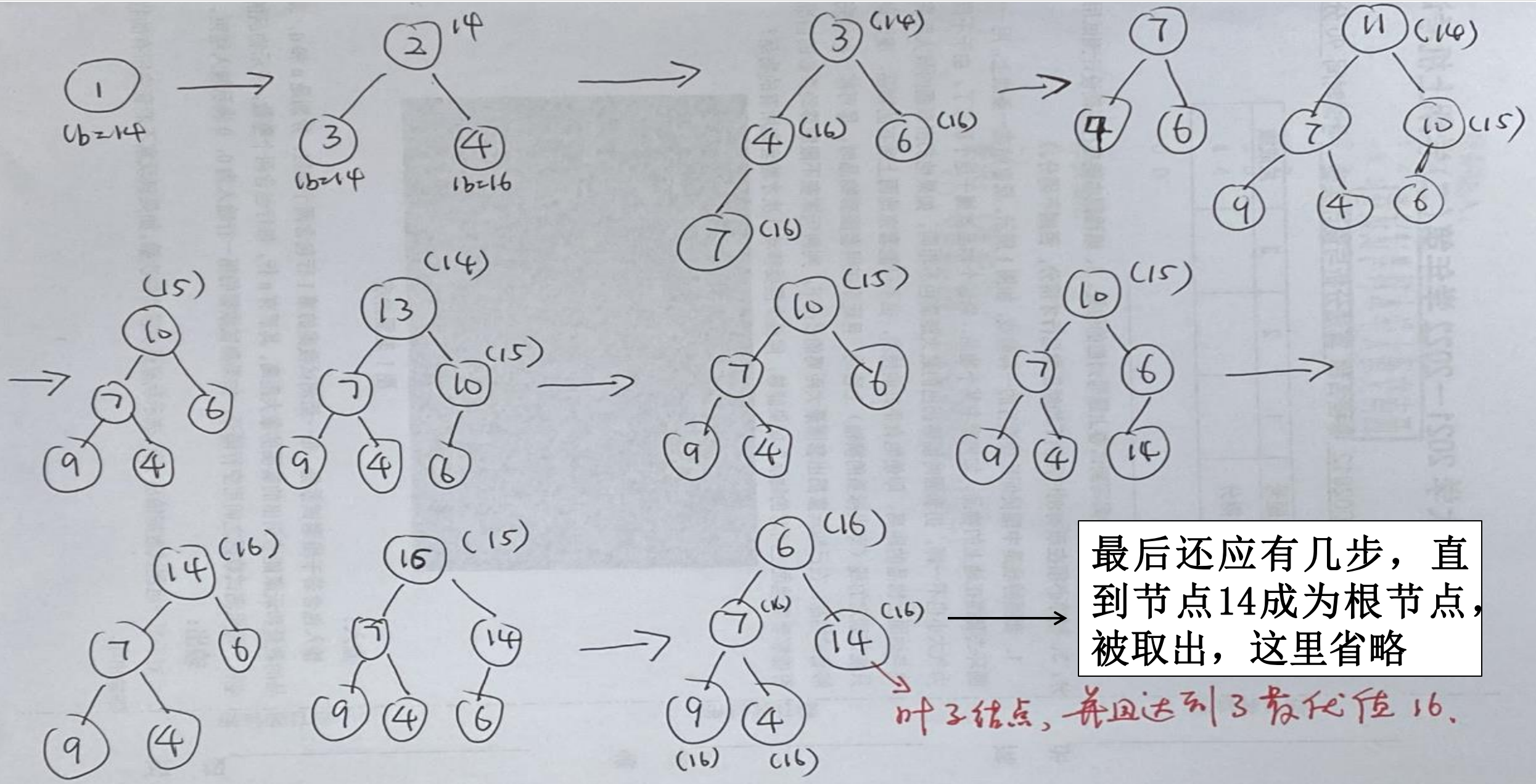
$s_j = j$ 行最小的元素＋已确定的边，表示城市 j 和确定的边相连。

例子的搜索空间



节点14是一个可行解，但其 lb 为 16，并不是活结点表中的最小值，所以节点 10 要继续扩展

（最小）堆的变化过程


1
lb=14
2
14
3
lb=14
4
lb=16
3
(14)
4
(16)
6
(16)
7
(16)
7
11
(16)
7
10
(15)
9
4
6
(15)
10
7
6
9
4
(14)
13
7
10
(15)
9
4
6
(15)
10
7
6
9
4
14
(16)
14
7
6
9
4
(15)
16
7
14
9
4
6
(16)
6
7
(16)
14
(16)
9
(16)
4
(16)
叶子结点，并且达到了变化值16.
最后还应有几步，直到节点14成为根节点，被取出，这里省略

旅行商问题（续）

```
while(heapLength > 0)
{
    HeapNode temp;
    temp = heap[1];
    level = temp.level;
    DeleteHeap(heap, heapLength);
    //堆顶元素是否是最终解
    if ((level == N - 1)
        &&C[temp.route[level] - 1][temp.route[0] - 1]!= INFINITY
        &&(temp.v + C[temp.route[level] - 1][temp.route[0] - 1])
                        <= MinDownBound(heap, heapLength))
    {
        for (i = 0 ; i < N ; i ++)
            输出 temp.route[i];
        输出 temp.route[0];
        输出 temp.v + C[temp.route[level] - 1][temp.route[0] - 1];
        return ;
    }
```

旅行商问题（续）

```
else

{
for(j = 1;j <= N;j++)
    {
        if(C[temp.route[level] - 1][j - 1]!= INFINITY&&!Contain(te
        {
            temp.route[level + 1] = j;
            temp.level = level + 1;
            float lb = ((temp.v + C[temp.route[level] - 1][j - 1]) *
                        + SiSum(C,temp) + SjSum(C,temp))/2
            if(lb <= upBound)
            {
                HeapNode node;
                node.v = temp.v + C[temp.route[level] - 1][j - 1];
                node.bound = lb;
                for(i = 0;i <= level;i++)        //在
                    node.route[i] = temp.route[i];
                node.route[level + 1] = j;
                node.level = level + 1;
                InsertHeap(heap,node,heapLength);
            }
        }
    }
```

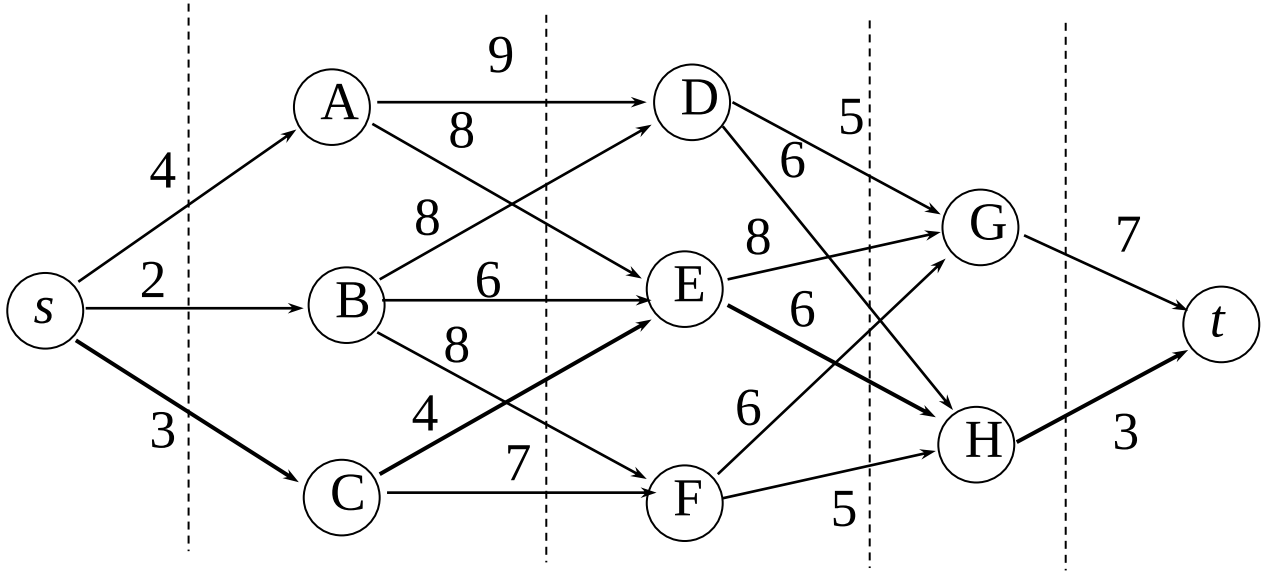
约束条件

扩展堆顶元素temp的后代

lb = 已经确定的边的长度+当前路径首尾两个节点的剩下一条边的下界+尚未经过的节点的边长下界

4.4 多段图最短路径

问题描述及分析



分析：采用优先队列式分支限界法，并用一最小堆来存储活结点表。

多段图最短路径（续）

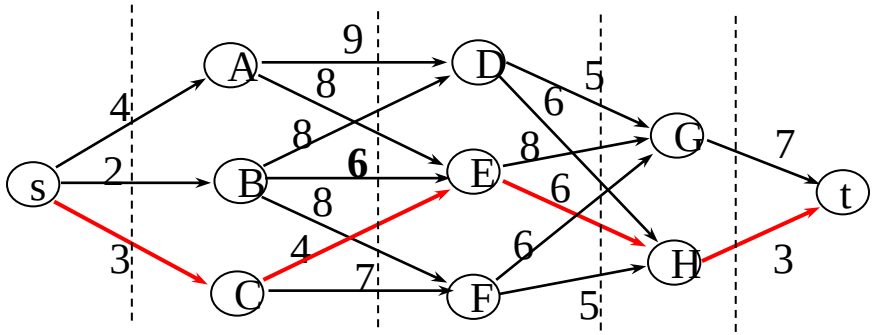
- 解向量：$X=(s, x_2, \ldots, t)$，s和t分别为起点和终点
- 约束条件：
  - 显式：$x_i=A, B, \ldots \ (i=2, \ldots, n)$
  - 隐式：$c_{ij}\neq\infty$
- 目标函数：$cost(i)=min\{c_{ij}+cost(j)\}$ ($i\leq j\leq n$且顶点j是i的邻接点)
  - 下界：把每一段最小的代价相加，2+4+5+3=14
  - 上界：2+7+6+3=18 (s→B→E→H→t)
- 下限界函数：假设已经确定了i段($1\leq i\leq k$)，其路径为$(r_1, r_2, \ldots, r_i, r_{i+1})$

$$db=\sum_{j=1}^{i} c[r_j][r_{j+1}]+\min_{<r_{i+1},v_p>\in E}\{c[r_{i+1}][v_p]\}+\sum_{j=i+2}^{k}\text{第}j\text{段的最短边}$$

与顶点r_{i+1}相连的边中，代价最小的边

（剩余顶点能够达到的最小代价）

多段图最短路径（续）



目标函数范围：[14, 18]

1 *start* *db*=14

2 s→A ***db*=20** **(4+8+(5+3))** ×

3 s→B ***db*=16** **(2+6+(5+3))**

4 s→C ***db*=15** **(3+4+(5+3))**

7 B→D *db*=18

8 B→E *db*=17

9 B→F *db*=18

5 C→E *db*=16

6 C→F *db*=18

10 E→G *db*=22 ×

11 E→H *db*=16

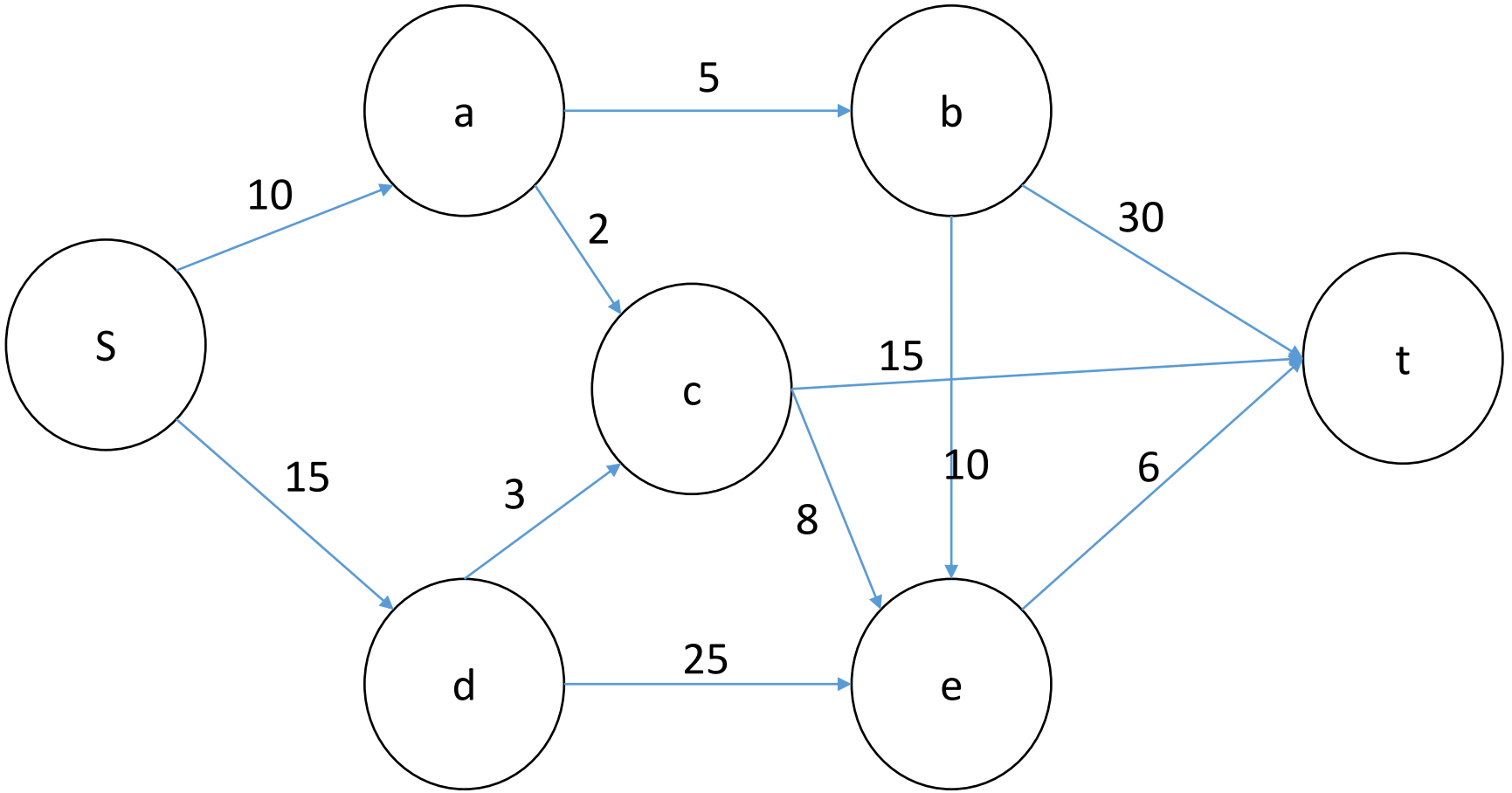
可行解，并且其值比其他下界都小

c=16

s→C→E→H→t

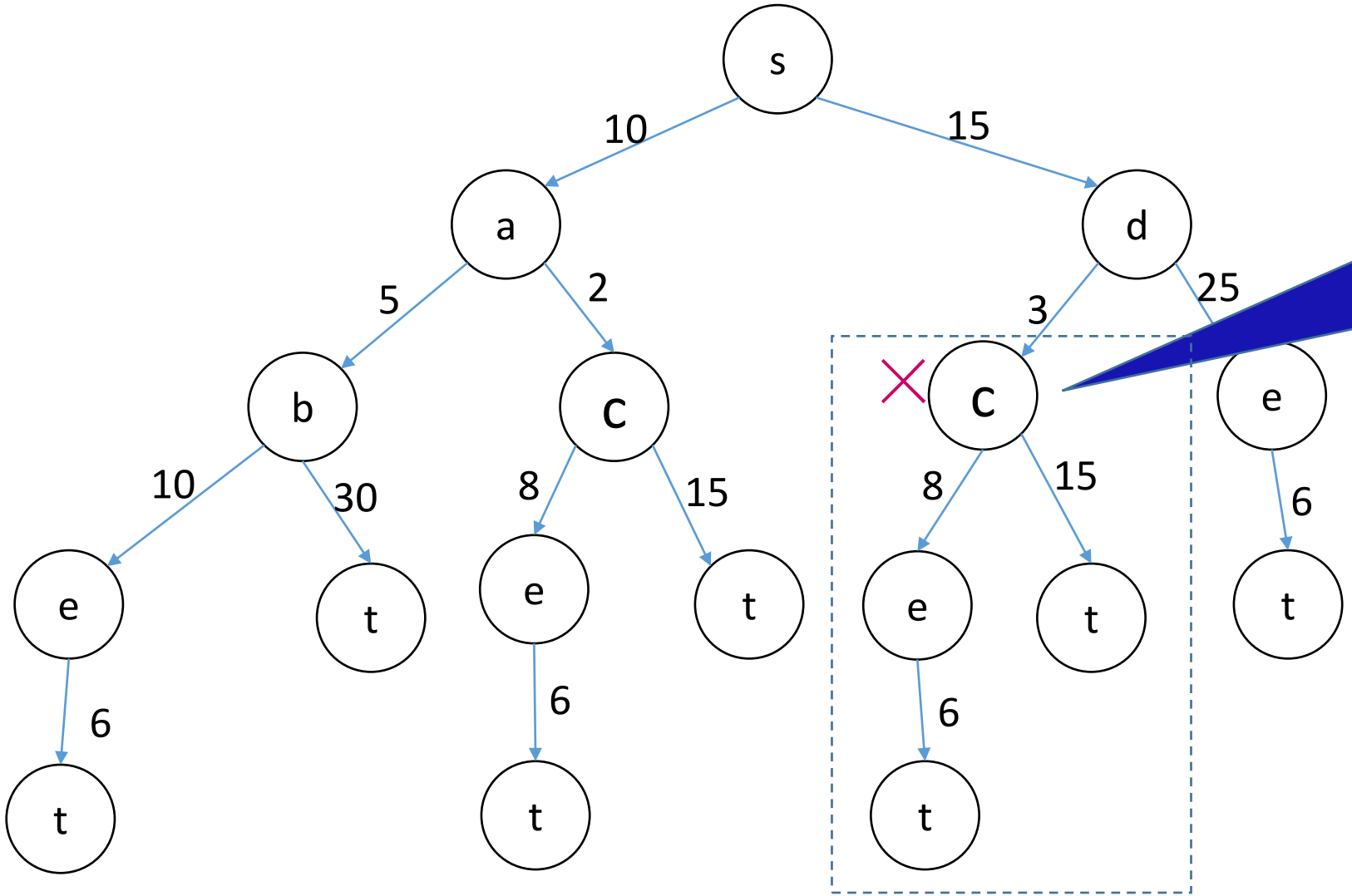
4.5 单源最短路径

- 针对下面的有向图，求从起点 s 到 t 的最短路径：



单源最短路径（续）

- 上述实例对应的解空间树如下图所示：

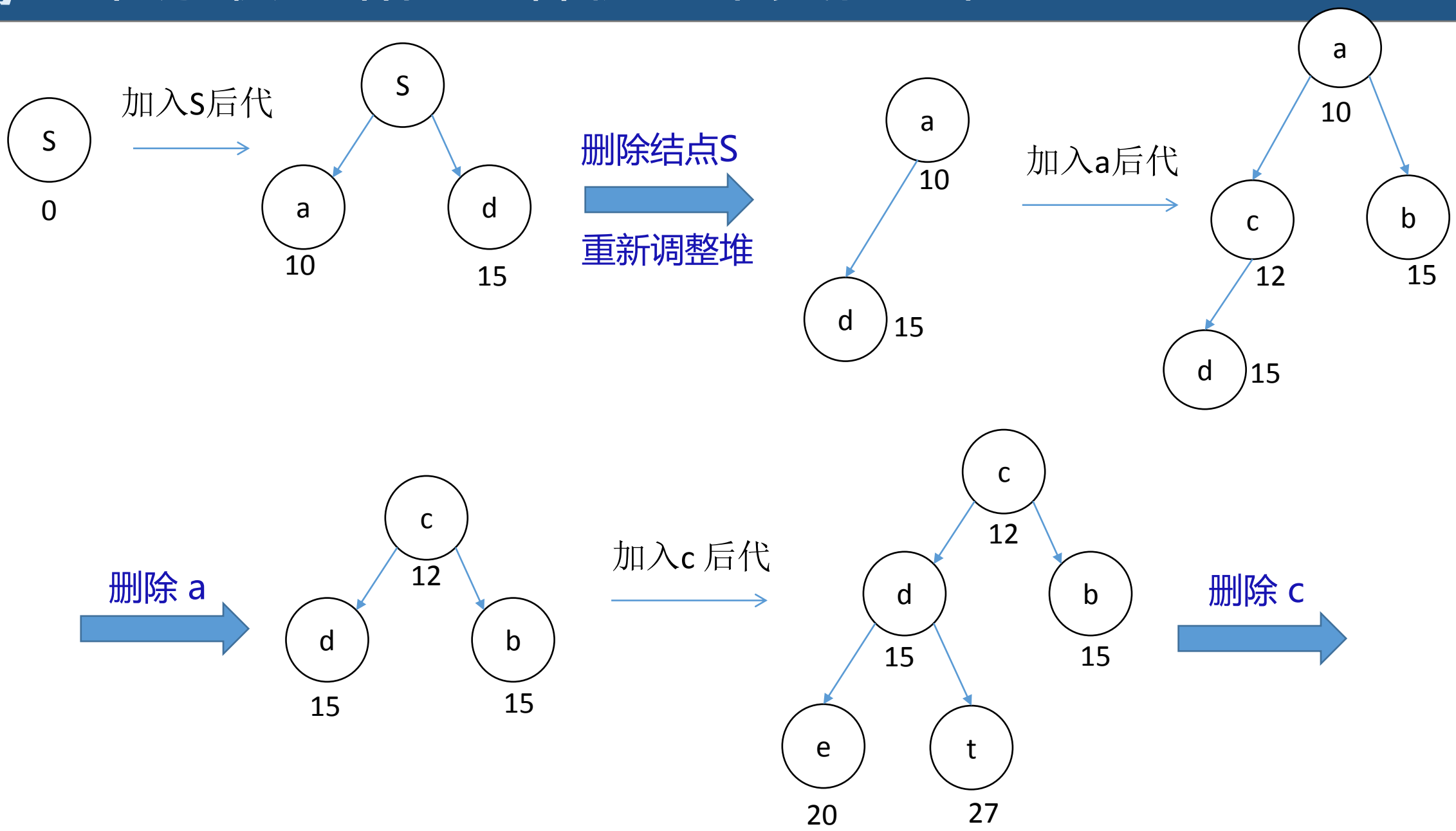


到达同一节点（如C）的路径可能有多条，只需保留最短路径，比最短路径长的可以剪枝

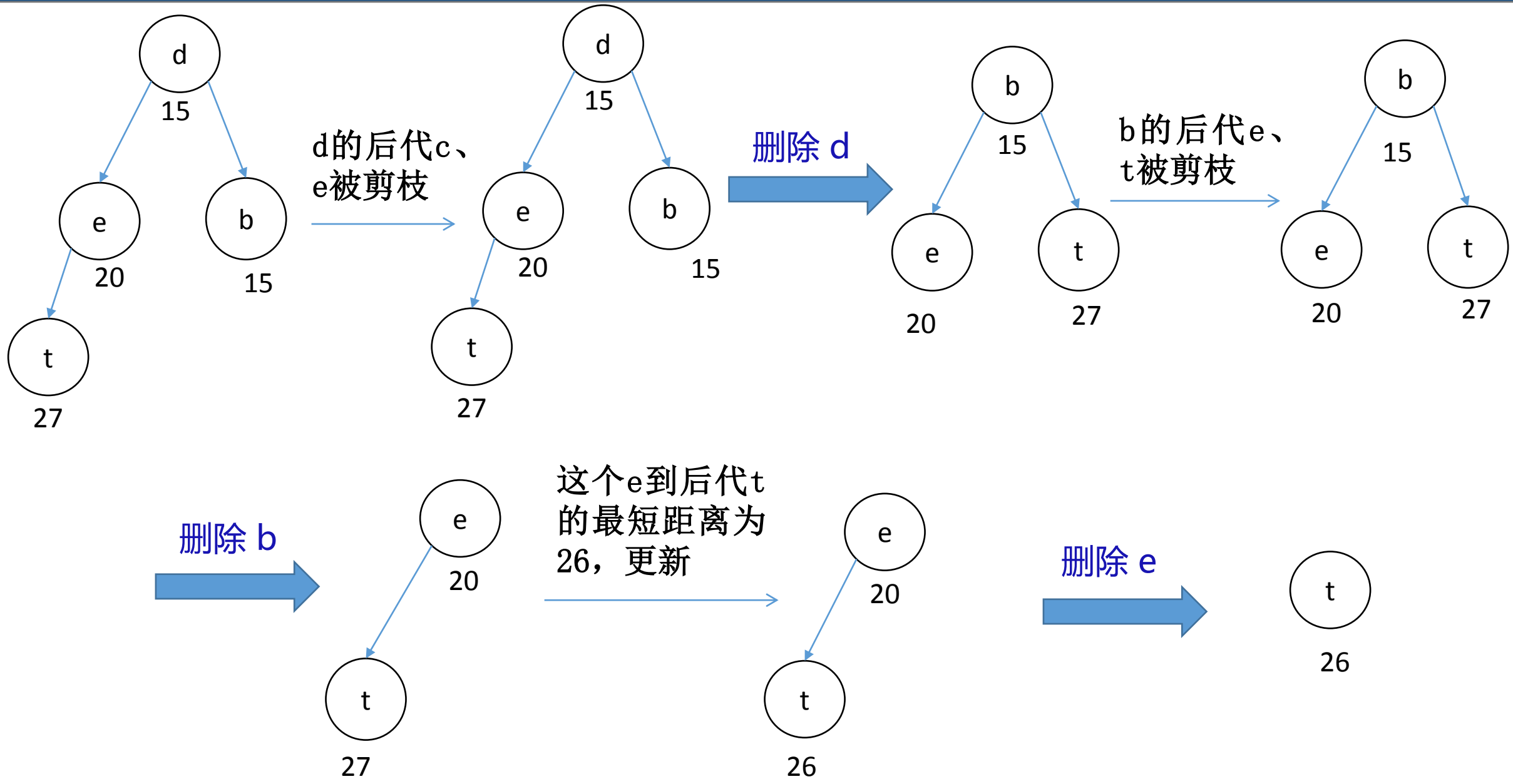
单源最短路径（续）

- 约束条件
  - 显然从起点到达图中同一顶点的路径之和越短，其结果也就越符合求解要求，反之越长的结点所对应的子树也就应该从解空间树中剪去（利用结点间的控制关系进行剪枝）。
- 限界函数
  - 从起点到当前顶点的最短路径长度（即只等于已经走过的最短路径长度）。
- 采用最小堆（小顶堆）
  - 采用最小堆存储活结点表，并在求解过程中不断的构建堆，直到处理完所有的结点。

单源最短路径—样例的堆变化过程



单源最短路径—样例的堆变化过程（续）



单源最短路径参考代码

```
//E.i=起点编号；E.length=0；dist[起点编号]=0；
while (true) {
    for (int j = 1; j <= n; j++)
        if ((c[E.i][j]<inf)&&(E.length+c[E.i][j]<dist[j])) {
            // 扩展 E.i 的后代（顶点 i 到顶点 j 可达，且满足控制约束）
            dist[j]=E.length+c[E.i][j];
            prev[j]=E.i;
            MinHeapNode<Type> N; // 新的活结点
            N.i=j;
            N.length=dist[j];
            H.Insert(N);   //将活结点N加入H（最小堆）
        }
    try {  H.DeleteMin(E);  }        // 取下一扩展结点
    catch (OutOfBounds) {  break;  } // 优先队列空
}
```

dist: 最短距离数组
prev: 前驱顶点数组
E： 当前的扩展节点
c: 邻接矩阵
H：最小堆

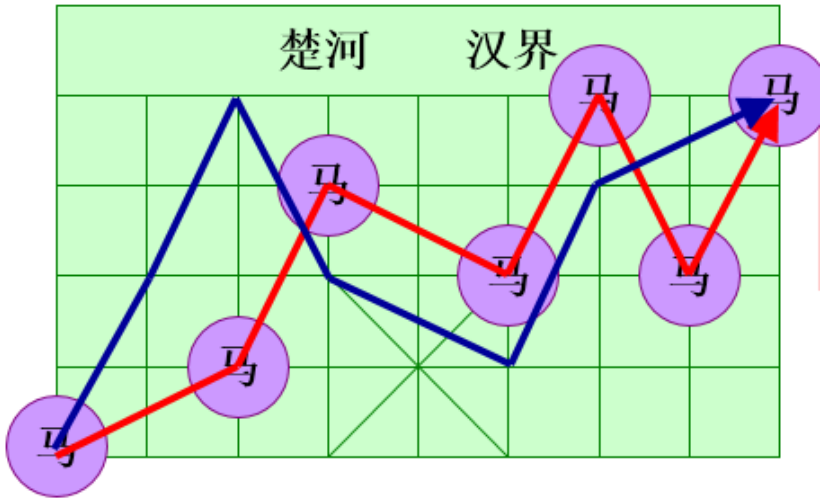
4.6 棋盘类问题小结

- 设棋盘有 n 行 m 列，用map[][]保存；map[][]中的“#”表示障碍，“.”表示通路。
- 棋子的行走方向包括：
  - 1、4个方向（上、下、左、右）；
  - 2、8个方向（上、下、左、右、左上、右上、左下、右下）；
  - 3、其他。
- 典型约束函数：
  - x>=1 && x<=n && y>=1 && y<=m && map[x][y]=“.”（x、y是棋子行、列号）
  - 如果在一次遍历中一个格子只能走一次，还需要设置visited数组。
- 求最少步数类问题的限界函数（EndX、EndY是出口坐标）：
  - 限界函数=当前已经走过的步数+abs(x-EndX)+abs(y-EndY)（x、y是棋子行、列号）
- 典型目标函数：
  - 包括到从初始位置到出口的最短步数、路径条数、环游所有点的路径条数，等等。

课堂练习：中国象棋马的棋盘遍历问题

在n＊m的棋盘中，马只能走“日” 字。马从位置(x,y)处出发，把棋盘的每一格都走一次，且只走一次。找出所有路径。

- 问题1：问题解的搜索空间？
  - 棋盘的规模是n*m，是指行有n条边，列有m条边。
  - 马在棋盘的点上走，所以搜索空间是整个棋盘上的n*m个点。
  - 用n*m的二维数组记录马行走的过程，初值为0表示未经过。

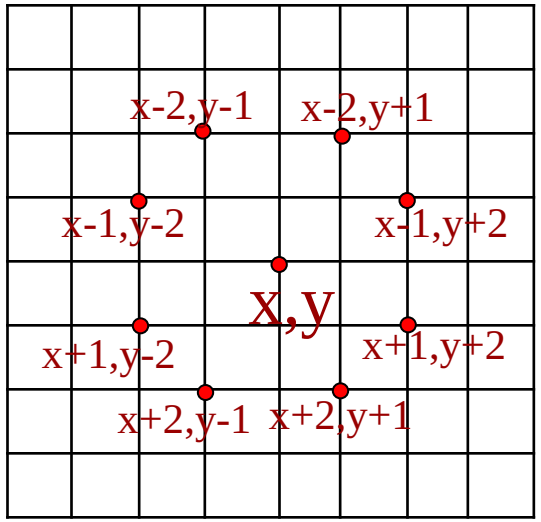


课堂练习：中国象棋马的棋盘遍历问题

在n＊m的棋盘中，马只能走“日” 字。马从位置(x,y)处出发，把棋盘的每一格都走一次，且只走一次。找出所有路径。

- 问题2：在寻找路径过程中，活结点的扩展规则？
  - 对于棋盘上任意一点A(x,y)，有八个扩展方向：

A(x+1,y+2),A(x+2,y+1)
A(x+2,y-1),A(x+1,y-2)
A(x-1,y-2),A(x-2,y-1)
A(x-2,y+1),A(x-1,y+2)



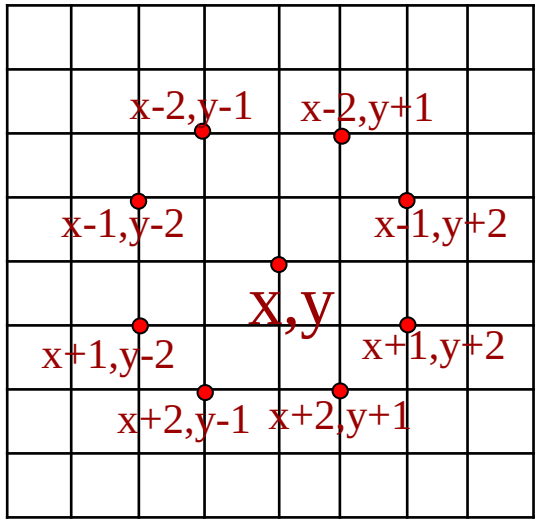
  - 为构造循环体，用数组fx[8]={1,2,2,1,-1,-2,-2,-1}, fy[8]= {2,1,-1,-2,-2,-1,1,2}来模拟马走“日”时下标的变化过程。

课堂练习：中国象棋马的棋盘遍历问题

在n＊m的棋盘中，马只能走“日” 字。马从位置(x,y)处出发，把棋盘的每一格都走一次，且只走一次。找出所有路径。

- 问题3：扩展的约束条件？
  - 不出边界；
  - 每个点只经过一次。

棋盘点对应的数组元素初值为0，对走过的棋盘点的值置为所走步数，起点存储“1”，终点存储“n*m”。



- 函数check，检查当前状态是否合理。

课堂练习：中国象棋马的棋盘遍历问题

在n＊m的棋盘中，马只能走“日” 字。马从位置(x,y)处出发，把棋盘的每一格都走一次，且只走一次。找出所有路径。

- 问题4：搜索解空间？

搜索过程是从任一点（x,y）出发，按深度优先的原则，从8个方向中尝试一个可以走的棋盘点，直到走过棋盘上所有n＊m个点。用递归算法易实现此过程。

注意问题要求找出全部可能的解，就要注意回溯过程的清理现场工作，也就是置当前位置为未经过。

课堂练习：中国象棋马的棋盘遍历问题

```
int  n=5 , m=4, dep , i, x , y , count;
int  fx[8]={1,2,2,1,-1,-2,-2,-1},fy[8]={2,1,-1,-2,-2,-1,1,2},a[n][m];
main( )
{ count=0;  dep=1;
  print('input x,y');      input(x,y);
  if (y>n  or  x>m  or  x<1  or  y<1)
       { print('x,y error!');   return;}
  for(i=1;i<=n;i++)
    for(j=1;j<=m;j++)
         a[i][j]=0;
  a[x][y]=1;
  find(x,y,2);
  if  (count=0 )    print("No answer!");
  else   print("count=",count);
}
```

数据结构定义：

1) 用一个变量dep记录递归深度，也就是走过的点数，当dep=n＊m时，找到一组解。

2) 用n＊m的二维数组a记录马行走的过程，初始值为0表示未经过。搜索完毕后，起点存储的是“1”，终点存储的是 “n＊m”。

课堂练习：中国象棋马的棋盘遍历问题

```
find(int x,int y,int dep)
{ int  i,xx,yy;
  for (i=1;i<=8;i++)      //加上方向增量,形成新的坐标
  { xx=x+fx[i];  yy=y+fy[i];
    if (check(xx,yy)=1) //判断新坐标是否出界,是否已走过
     { a[xx,yy]=dep;      //走向新的坐标
       if (dep=n*m)  output( );
       else
          find(xx,yy,dep+1);    //从新坐标出发,递归下一层
       a[xx,yy]=0;           //回溯,恢复未走标志
     }
  }
}
```

```
output( )
{     count=count+1;
      print(“换行符”);
        print(“count=”,count);
        for (x=1;x<=n;i++)
        { print(“换行符”);
          for (y=1;y<=m;y++)
              print(a[x,y]);
          }
}
```

本章小结

- 一、回溯法概述
- 二、常见的回溯法算法
- 三、分支限界法
- 四、常见的分支限界算法